

多烟幕干扰弹投放优化设计

摘 要

利用无人机投放烟幕干扰弹的技术已成为现代战争中的常用手段，而恰当的烟幕干扰弹的投放策略可有效对来袭武器实现干扰效果。本文通过考虑烟幕干扰弹对目标的有效遮蔽时间问题，对多烟幕干扰弹的投放策略进行优化设计。

针对问题一：对于一种给定的投放策略，首先基于运动学知识对导弹的运动、无人机的运动以及起爆前后烟幕干扰弹的运动进行研究，得到其每一时刻的位置坐标；接着依据导弹是否在烟幕云团内部分情况建立**有效遮蔽模型**。对于导弹在云团外的情况，我们在定义了遮蔽条件后对真目标进行**离散化**处理，之后为方便写出遮挡圆锥的方程建立**圆锥坐标系**，最后确定出有效遮挡判定条件并利用**坐标变换**对离散化后的真目标进行判断。同时我们建立了有效遮蔽时刻的集合进而显式化了**有效遮蔽时长**的表达式，通过遍历烟幕云团存在的时间，得到有效遮挡的每一时刻从而计算出**有效遮蔽时长** $t_{eff}=1.391s$ 。

针对问题二：我们建立了以无人机飞行方向、飞行速度、烟幕干扰弹投放点、起爆点为决策变量，有效遮蔽时长最大为目标的**单目标优化模型**。我们逐个对决策变量的约束条件进行分析，并基于问题一的单无人机投放模型与有效遮蔽模型进行修正。由于决策变量较多且范围较大，我们使用**粒子群算法**进行单目标优化模型的求解，最终求得**最长有效遮蔽时间**为 $T_{eff}=4.58s$ ，详细参数见正文表 2。

针对问题三：构建多烟幕干扰弹下有效遮蔽单目标优化模型。我们首先对单无人机多干扰弹运动情况进行分析；接着结合集合的思想，通过对任意时刻烟幕云团出现的个数对时间分为三个阶段并建立**每个阶段下有效遮蔽的判定条件**；通过写出约束条件最终得到以无人机飞行方向、飞行速度、每个干扰弹投放点及起爆点为决策变量的单目标优化模型，同时注意到同一架无人机**投放时间间隔的合理性**约束。利用粒子群算法进行优化求解，得到该条件下的最优投放策略，**最大有效遮蔽时长** $T_{eff}=6.41s$ ，其余参数详见 result1.xlsx。

针对问题四：在问题三多烟幕云团遮蔽模型建立的基础上，我们建立以三架无人机飞行方向、飞行速度、烟幕干扰弹投放点及起爆点为决策变量，有效遮蔽时长最长为目标函数的多无人机下有效遮蔽的单目标优化模型，由于决策变量数量较多，同样采用粒子群算法进行求解得到**最大有效遮蔽时长** $T_{eff}=11.53s$ ，结果详见 result2.xlsx。

针对问题五：我们将五架无人机分别应用问题三中模型得到五个有效遮蔽时长，以他们相加之和作为新的目标函数建立单目标优化模型，最终求得该情况下**最大有效遮蔽时长** $T_{eff}=25.9s$ ，结果详见 result3.xlsx。

关键词：烟幕干扰弹 有效遮蔽时长 单目标优化 粒子群算法 圆锥坐标系

一、问题重述

1.1 问题背景

近年来,烟幕干扰技术不断发展^[1],运用成本低、长续航、风险小的无人机投放烟幕干扰弹在来袭武器与保护目标之间形成遮挡的方法已成为现代战争中的常用技术。烟幕干扰弹脱离无人机后在重力作用下运动,一段时间后起爆形成持续时间 10s、有效半径为 10m 的烟幕云团^[2],通过采用特定技术使云团以固定速度速度匀速下沉。而如何设计投放策略,使多枚烟幕干扰弹的有效遮蔽时间最长成为应用中的核心问题。

1.2 问题提出

来袭武器为飞行速度为 300m/s 的空地导弹,真实攻击目标为一圆柱体,为掩护真实目标另设置一个假目标,将假目标所在位置视为坐标原点,水平地面为 xy 平面建立三维坐标系,真实目标下地面圆心坐标为 $(0, 200, 0)$ 。当警戒雷达发现导弹时,携有烟幕干扰弹的无人机可瞬时调整一次飞行方向,并以 $70 - 140\text{m/s}$ 的某一速度等高度匀速直线飞行。已知发现导弹时各导弹的位置和无人机位置坐标,和烟幕干扰弹的遮蔽原理,为实现更有效的烟幕干扰效果,对其投放策略进行优化设计,我们建立数学模型解决以下问题:

问题一:考虑仅有一枚来袭导弹和一架无人机 FY1 并只投放 1 枚烟幕干扰弹,发现导弹 M1 后调整 FY1 飞行速度为 120m/s ,并在所在高度平面上朝着假目标方向飞行。在飞行 1.5s 后投放 1 枚烟幕干扰弹,并在 3.6s 后起爆,计算出烟幕干扰弹对 M1 的有效遮蔽时长。

问题二:同样考虑仅有一枚来袭导弹和一架带有 1 枚烟幕干扰弹的无人机 FY1,为使得其对 M1 的有效遮挡时间最长,设计 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点,在满足一定约束的前提下让有效遮蔽时长达到最大。

问题三:若 FY1 投放 3 枚烟幕干扰弹对 M1 进行干扰,投放间隔不少于 1s,设计 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点使得有效遮蔽时长达到最大。

问题四:考虑 3 架无人机 FY1, FY2, FY3, 每架无人机各投放 1 枚烟幕干扰弹对 M1 进行干扰,设计这 3 架无人机的最佳投放策略。

问题五:考虑 5 架无人机且每架无人机至多可投放 3 枚烟幕干扰弹,对导弹 M1, M2, M3 进行干扰,对 5 架无人机的烟幕干扰弹投放策略进行优化设计,使多枚烟幕干扰弹对真目标的有效遮蔽时间最长。

二、模型假设

1. 假设飞行环境为理想环境，忽略气象因素、地形障碍等。
2. 假设多导弹独立飞行，相互无协同且互不相撞。
3. 假设烟雾干扰弹在起爆后瞬时形成半径大于等于 10 米的烟雾云团。
4. 假设假目标可视作无形状无大小的质点。

三、符号说明

符号	说明	单位
M_i	第 i 枚导弹的位置坐标	
$v_{M,i}$	第 i 枚导弹的速度	m/s
$\vec{e}_{M,i}$	第 i 枚导弹的单位方向向量	
P_{it}^j	第 i 架无人机投放第 j 枚干扰弹时刻的位置坐标	
$P_{it,bomb}^j$	第 i 架无人机投放的第 j 枚干扰弹起爆时刻的位置坐标	
$t_{it,rel}^j$	第 i 架无人机投放第 j 枚干扰弹的时刻	s
$t_{it,bom}^j$	第 i 架无人机投放的第 j 枚干扰弹起爆的时刻	s
v_{sink}	烟雾云团下沉速度	m/s
R	烟雾云团的半径	m
α	遮挡圆锥的半顶角	$^\circ$
h_{goal}	真目标圆柱的高度	m
Ω	目标离散化后多所有点构成的集合	
Λ	有效遮蔽时刻集合	
t_{teff}	有效遮蔽时长	s
T_{teff}	最大有效遮蔽时长	s
$\vec{e}_{fi}$	第 i 架无人机飞行单位方向向量	
γ_i	第 i 架无人机飞行方向与 x 轴正方向的夹角（逆时针为正）	$^\circ$

四、问题分析

4.1 问题一的分析

在问题一中，对于一种固定投放策略，即已知无人机飞行方向、飞行速度、烟雾干扰弹投放点、烟雾干扰弹起爆点对烟雾干扰弹的有效遮蔽时长进行分析。首先建立单无人机投弹模型，对导弹运动以及起爆前的烟雾干扰弹的运动进行分析。接下来建

立起有效遮蔽模型，详细分析两种情况的有效遮蔽，并通过坐标变换得到遮挡圆锥面的方程，再通过离散化处理真目标构建遮挡判定条件，得到某一时刻为有效遮蔽时刻的充要条件，最终通过遍历烟幕云团存在时间计算出有效遮蔽时长。同时对无人机飞行速度烟幕干扰弹投放点、起爆点对模型进行灵敏度分析。

4.2 问题二的分析

在问题二中，我们建立以无人机飞行方向、飞行速度、烟幕干扰弹投放点、起爆点为决策变量，有效遮蔽时长最大为目标的单目标优化模型。通过对决策变量的约束条件进行分析，并基于问题一的单无人机投弹模型与有效遮挡模型进行修正，最终利用粒子群算法对该复杂优化模型进行求解得到最大的有效遮挡时长。

4.3 问题三的分析

在问题三中，需要考虑单无人机投放多枚干扰弹情况下最优的投放策略，即需要确定无人机飞行方向、飞行速度、三枚烟幕干扰弹分别的投放点、起爆点，使得对导弹 M1 的有效遮蔽时长最大。我们首先建立单无人机多干扰弹的运动模型，计算出导弹运动轨迹、无人机运动轨迹以及干扰弹起爆前后的运动轨迹，并对投放位置与起爆位置进行确定。接着我们根据烟幕云团的个数将时间分为三个阶段，进而建立每个阶段下的有效遮挡判定条件并进行算法设计。最后使用粒子群算法对单目标优化模型进行求解得到最优投放策略。

4.4 问题四的分析

在问题四中，由于该种情况下任意时刻烟幕云团最多出现的个数仍为三个，我们基于问题三建立的多烟幕云团遮蔽模型进行分析，对于三架无人机各自的飞行方向，飞行速度，烟幕干扰弹投放时间、起爆时间共 12 个决策变量，分析该种情况的约束条件，从而建立多无人机下有效遮蔽的单目标优化模型，并使用粒子群算法进行求解得到该种情况下的最佳投放策略。

4.5 问题五的分析

在问题五中，考虑 5 架无人机每架无人机投放 3 枚烟幕干扰弹的情形。由于共 15 枚干扰弹以及 3 枚导弹，要考虑其有效遮挡时间相互重叠问题过于复杂，我们基于问题三中已建立的单无人机三个干扰弹的有效遮蔽单目标优化模型，对 5 架无人机分别使用一次该模型计算最长有效遮蔽时长，最终将 5 个有效遮蔽时长相加作为新的目标函数，接着分析该情形下的约束条件，进而建立单目标优化模型，继续利用粒子群算法求解得到的最佳投放策略。

五、模型的建立与求解

5.1 问题一模型的建立与求解

在问题一中，我们需要在已知无人机飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点的情况下对烟幕干扰弹的有效遮蔽时长进行分析。首先建立单无人机投弹模型，对导弹运动以及起爆前的烟幕干扰弹的运动进行分析。接着我们建立有效遮蔽模型，分析两种情况的有效遮蔽，并通过坐标变换、离散化处理真目标的方式建立遮挡圆锥面方程，构建遮挡判定条件。通过遍历烟幕存在的 20s 时长得到所有满足判定条件的时刻最终得到有效遮蔽时长。



图 1 问题一流程图

5.1.1 单无人机投弹模型的建立

为方便后续分析，首先我们建立以假目标为坐标原点，水平地面为 xy 平面的三维直角坐标系。真目标为一个半径 $7m$ 、高 $10m$ 的圆柱体，其底面圆心坐标为 $(0, 200, 0)$ 。

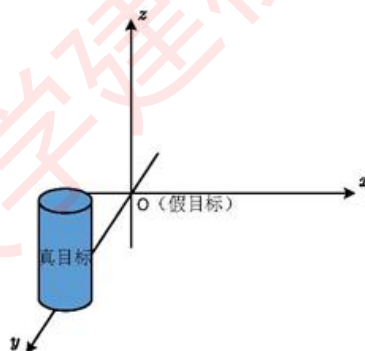


图 2 坐标系示意图

● 导弹运动分析

导弹 M1 初始位置坐标为 $M_{10}(20000, 0, 2000)$ ，以恒定速度 $v_{M1} = 300m/s$ 朝向原

点方向飞行，则其速度的单位方向向量可表示为：

$$\vec{e}_{M1} = \frac{-\overrightarrow{OM_{10}}}{|\overrightarrow{OM_{10}}|} \quad (1)$$

其中 $|\overrightarrow{OM_{10}}|$ 表示模长长度。则导弹在 t 时刻的位置 $M_1(x_{M1}, y_{M1}, z_{M1})$ 向量方程可表示为：

$$M_1(t) = M_{10} + v_{M1} \vec{e}_{M1} t, \quad t \geq 0 \quad (2)$$

● 无人机运动分析

已知无人机 FY1 在受领任务初始时刻的坐标为 $P_{10}(x_{10}, y_{10}, z_{10})$ ，在本问题中设定 FY1 的飞行速度为 $v_1 = 120m/s$ ，飞行方向为保持 z 轴坐标不变，朝原点方向等高度飞行。因此其飞行单位方向向量为：

$$\vec{e}_{F1} = \frac{-(x_{10}, y_{10}, 0)}{|(x_{10}, y_{10}, 0)|} \quad (3)$$

从初始时刻到投放时刻 $t_{1,rel} = 1.5s$ 这段时间内时，由于做匀速直线运动，无人机的运动方程为：

$$P_1(t) = P_{10} + v_1 \vec{e}_{F1} t, \quad 0 \leq t \leq t_{1,rel} \quad (4)$$

设 FY1 投放烟幕干扰弹时的坐标为 $P_{1t}(x_{1t}, y_{1t}, z_{1t})$ ，于是：

$$P_{1t} = P_{10} + v_1 \vec{e}_{F1} t_{1,rel} \quad (5)$$

● 烟幕干扰弹运动分析

Step1: 投放后的平抛运动

干扰弹在投放后，由于存在水平方向的初速度和竖直方向的重力加速度，作平抛运动。此时的水平速度恒与 FY1 飞行速度 v_1 保持一致，竖直方向为初速度为 0 的自由落体运动，其在这段时间内的运动方程为：

$$P_1(t) = P_{1t} - v_1 \vec{e}_x (t - t_{1,rel}) - \frac{1}{2} \vec{g} (t - t_{1,rel})^2, \quad t_{1,rel} \leq t \leq t_{1,bom} \quad (6)$$

其中 $\vec{e}_x = (1, 0, 0)$ ， $\vec{g} = (0, 0, g)$ ， g 为重力加速度，取 $9.8m/s^2$ 。由于投放后的干扰弹在 $\Delta t_{1,bom} = 3.6s$ 后起爆，其起爆时刻为 $t_{1,bom} = 1.5s + 3.6s = 5.1s$ ，因此其起爆时的位置 $P_{1,bomb}(x_{1b}, y_{1b}, z_{1b})$ 可写成参数方程：

$$\begin{cases} x_{1b} = x_{1t} - v_1 (t_{1,bom} - t_{1,rel}) \\ y_{1b} = y_{1t} \\ z_{1b} = z_{1t} - \frac{1}{2} g (t_{1,bom} - t_{1,rel})^2 \end{cases} \quad (7)$$

Step2: 起爆后的下沉运动

干扰弹在 t_{12} 时刻后起爆，瞬时形成半径 $R = 10m$ 的球形烟幕云团，水平方向速度消失，仅以 $v_{sink} = 3m/s$ 的速度沿竖直方向下沉。则在这段时间烟幕云团的运动方程

为:

$$P_1(t) = P_{1,born} - v_{sink} \vec{e}_z(t - t_{1,born}), \quad t \geq t_{1,born} \quad (8)$$

设其起爆后坐标为 $P_{1,after}(x_{1a}, y_{1a}, z_{1a})$, 参数方程可表示为:

$$\begin{cases} x_{1a} = x_{1b} \\ y_{1a} = y_{1b} \\ z_{1a} = z_{1b} - v_{sink}(t - t_{1,born}) \end{cases}, \quad t \geq t_{1,born} \quad (9)$$

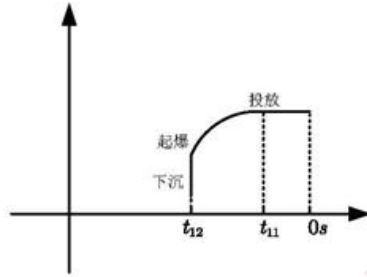


图 3 无人机投弹时间历程

5.1.2 有效遮蔽模型的建立

在 t_{12} 时刻后形成烟幕云团, 为判断该球形云团是否能完全遮住导弹对真目标的视野, 形成有效遮蔽, 我们建立几何模型进行判断。

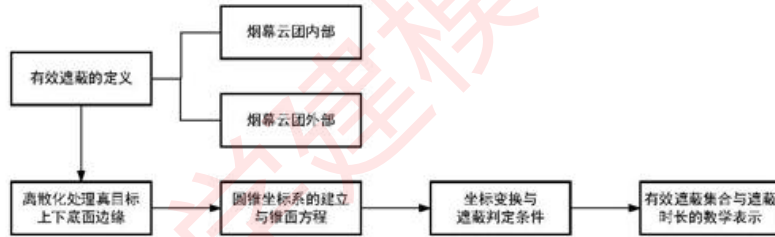


图 4 有效遮蔽模型建立流程

● 有效遮蔽的定义

Case 1

在任意时刻下, 如果导弹 M1 未进入烟幕云团内部, 即 $|M_1 P_{1,after}| > R$ 时, 球形烟幕云团对导弹的视野遮蔽在空间中可表示成一个母线无限延伸的圆锥, 即云团可遮挡在次圆锥内部物体的视野, 如下图所示:

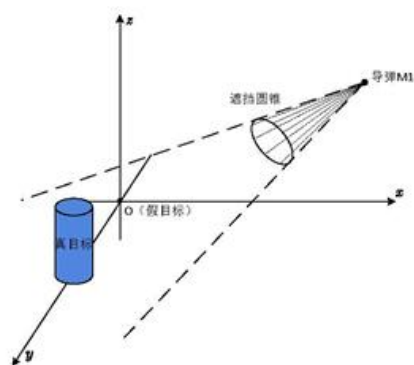


图 5 遮挡视野圆锥示意图

有效遮蔽是指在导弹飞行过程中烟幕云团可完全遮挡住导弹对真目标的视野。在空间中每一时刻以 M1 为顶点的射线与球形烟幕云团相切都构成一个圆锥，因此只要这个无限延长的圆锥能够完全包裹住真目标圆柱体即可形成有效遮蔽。从几何上看即导弹与烟幕云团的最上端的切线的延长线与真目标圆柱体的上表面无交点，导弹与烟幕云团最下端切线的延长线与真目标圆柱体的下表面无交点。

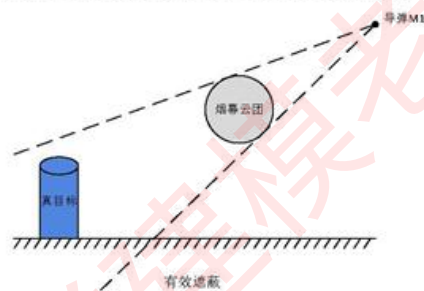


图 6 圆锥有效遮蔽示意图

下图表示两种无效遮蔽的情况，即圆锥不能够完全包裹住真目标圆柱体：

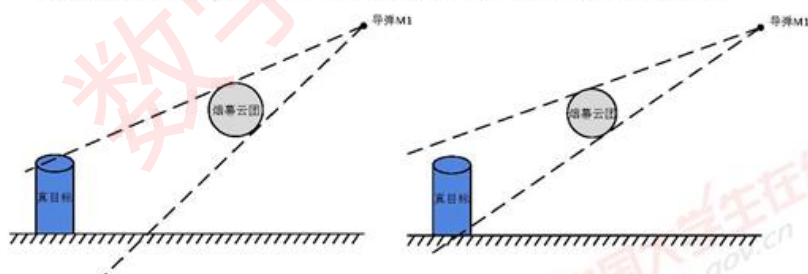


图 7 圆锥无效遮挡示意图

Case 2

如果导弹 M1 未进入烟幕云团内部, 即 $|M_1 P_{1,after}| \leq R$ 时, 导弹已经进入或接触到烟幕云团, 此时导弹的视线已经被完全阻碍无法发现真目标, 形成有效遮蔽。即满足该条件的所有时刻都算入有效遮蔽时长中。

因此接下来我们需要对第一种情况进行进一步分析。

● 真目标圆柱的离散化处理

由有效遮挡的定义, 当真目标圆柱上、下底面的圆边缘均在遮挡圆锥内部时整个圆柱即在遮挡圆锥的内部; 当整个圆柱在遮挡圆锥内部时显然圆柱的上下两个底面的原边缘均在整个圆锥内部。因此两个条件是等价的。

我们只需判断真目标圆柱上、下底面的圆边缘均在遮挡圆锥内部即可确定这一时刻为有效遮蔽。

由于上下底面为连续物体, 我们对上、下底面圆的边缘进行离散化处理。将上底面边缘按 θ 进行均分, 离散化为 $N=100$ 个点。同理下底面我们进行相同的操作, 并将所有 $2N$ 个点记为集合 $\Omega = \{A_k\}_{k=1}^N \cup \{B_k\}_{k=1}^N$ 。

进而上底面离散化点的坐标可表示为:

$$A_k(x_k^w, y_k^w, z_k^w) = (R \cos k\theta, R \sin k\theta + 200, h_{goal}), \quad k = 1, 2, \dots, N \quad (10)$$

其中 R 为烟幕云团的半径, $h_{goal} = 10m$ 为真目标圆柱的高。同理下底面的离散点可表示为:

$$B_k(x_k^b, y_k^b, z_k^b) = (R \cos k\theta, R \sin k\theta + 200, 0), \quad k = 1, 2, \dots, N \quad (11)$$

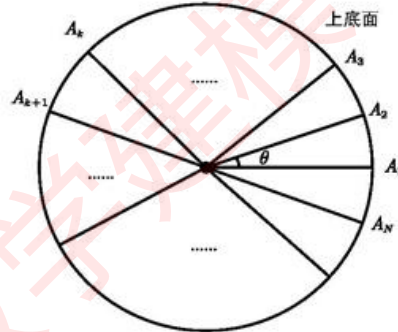


图 8 上底面离散化处理

在此处理下我们只需要判断这 $2N$ 个点的坐标是否均在圆锥面内部即可。

● 圆锥坐标系的建立与遮挡圆锥面方程

为进行上述判断我们还需要建立圆锥面的方程。在上部分定义的遮挡圆锥顶点与轴线方向并不规则, 在原坐标系下表示较为复杂, 因此我们建立一个新的圆锥坐标系,

并在新坐标系下表示出遮挡圆锥面的方程。

以任意时刻导弹的位置 $M_1(x_{M1}, y_{M1}, z_{M1})$ 为新坐标原点, M_1 与烟幕云团球心的连线为新 z 轴方向, 并取与新 z 轴方向垂直且相互垂直的两个方向为新 x 、 y 方向建立圆锥坐标系:

1. 确定并单位化新坐标系 z 轴方向:

$$u_z = \frac{\overrightarrow{M_1 O_1}}{|\overrightarrow{M_1 O_1}|} \quad (12)$$

其中 $|\overrightarrow{M_1 O_1}| = \sqrt{(x_{M1} - x_{1a})^2 + (y_{M1} - y_{1a})^2 + (z_{M1} - z_{1a})^2}$ 为导弹到烟幕圆心的距离。

2. 构建新坐标系下标准正交基: 记原坐标系下标准正交向量为 $e_1 = (1, 0, 0)$, $e_2 = (0, 1, 0)$, $e_3 = (0, 0, 1)$ 。选择辅助向量 α 为 e_1 、 e_2 、 e_3 中与 u_z 点乘绝对值最小的一个。于是:

$$\begin{cases} u_x = \frac{\alpha \times u_z}{|\alpha \times u_z|} \\ u_y = u_z \times u_x \end{cases} \quad (13)$$

即得到新坐标系下 x' 、 y' 、 z' 的方向。

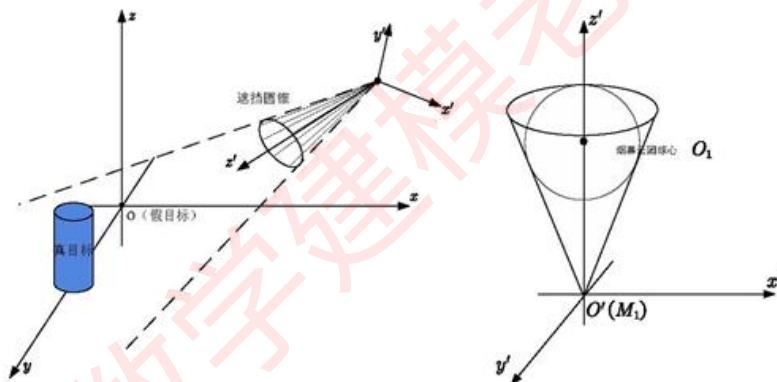


图 9 圆锥坐标系示意图

导弹到球心的距离为 $|\overrightarrow{M_1 O_1}|$, 且随着时间 t 而变动。设圆锥半顶角为 $\alpha = \alpha(t)$, 由于圆锥与烟幕云团相切, 由几何关系:

$$\alpha(t) = \arcsin \frac{R}{|\overrightarrow{M_1 O_1}|} \quad (14)$$

设圆锥面上一点为 (x', y', z') , 则有:

$$\tan \alpha(t) = \frac{\sqrt{(x')^2 + (y')^2}}{z'} \quad (15)$$

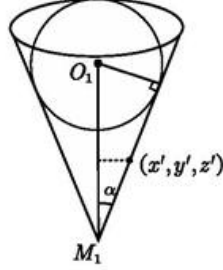


图 10 圆锥几何示意图

综上得到新坐标系下圆锥面方程：

$$(x')^2 + (y')^2 = (z')^2 \tan^2 \alpha, \quad z' \geq 0 \quad (16)$$

● 坐标系转化与遮蔽判定

针对两个坐标系，我们首先建立模型进行坐标转换。设旧坐标系下一点 $Q(x, y, z)$ ，其对应新坐标系下的点为 $Q'(x', y', z')$ ，已知任一时刻导弹位置为 $M_1(x_{M1}, y_{M1}, z_{M1})$ ，于是坐标变换可表示为：

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = R(t) \begin{pmatrix} x - x_{M1} \\ y - y_{M1} \\ z - z_{M1} \end{pmatrix} \quad (17)$$

其中 $R = R(t)$ 表示从旧坐标系到新坐标系的旋转矩阵，由空间解析几何知识接我们可以写出旋转矩阵：

$$R = (u_x, u_y, u_z)^T \quad (18)$$

接下来，我们对离散化处理的点进行是否遮蔽的判定：

1. 任取集合 Ω 中一点 $D_k(x_k, y_k, z_k)$ ，将其坐标变换到新坐标系下 $D'_k(x'_k, y'_k, z'_k)$ 。
2. 在新坐标系下写出判定条件：若 D'_k 在圆锥内部，则有：

$$(x'_k)^2 + (y'_k)^2 \leq (z'_k)^2 \tan^2 \alpha, \quad z'_k \geq 0 \quad (19)$$

即：

$$\begin{cases} z'_k \geq \frac{\sqrt{(x'_k)^2 + (y'_k)^2}}{\tan \alpha} \\ z'_k \geq 0 \end{cases} \quad (20)$$

基于上述分析，我们遍历 t 时刻时集合 Ω 中所有的点，当所有的点均满足 (20) 式的判定条件时，记这一时刻为有效遮蔽时刻，并记入有效遮蔽时长中。为表示有效遮蔽时长，设有效遮蔽时刻集合：

$$A = \left\{ t_i \left| \begin{cases} z'_k \geq \frac{\sqrt{(x_k')^2 + (y_k')^2}}{\tan \alpha(t_i)}, k=1 \dots 2N \text{ or } |O_1 M_i| \leq R \\ z'_k \geq 0 \end{cases} \right. \right\} \quad (21)$$

其中 t_i 为从起爆 $t_{1,bom}$ 到烟幕云团消失 $t_{1,bom} + 20s$ 的时间段内某一时刻，设示性函数：

$$\chi(t_i) = \begin{cases} 1, & t_i \in A \\ 0, & t_i \notin A \end{cases} \quad (22)$$

于是有效遮蔽时长可表示为：

$$t_{eff} = \sum_{i=1}^K \chi(t_i) \Delta t \quad (23)$$

其中 K 为遍历的时间点个数， Δt 为小时间间隔。

结合上节单无人机投弹模型

5.1.3 模型的求解

综合上述分析，我们通过遍历烟幕干扰弹存在的 20s 时间，并判断满足两种情况的有效遮蔽的时刻，算法步骤如下：

算法步骤

Step1: 时间步长分割

对从起爆到烟幕云团消失的时间段 $[t_{1,bom}, t_{1,bom} + 20]$ ，以步长 $\Delta t = 0.001s$ 进行分割，共计 K 个时间点，并进行遍历。

Step2: 运动状况分析

对分割后的每个时间点 t_i ，利用运动模型计算出导弹、无人机以及烟幕干扰弹的在该时刻的位置坐标。

Step3: 判断导弹是否位于烟幕云团内

比较每一时间点 $|M_1 O_1|$ 与 R 的大小，若 $|M_1 O_1| \leq R$ ，则将该时间点放入集合 A 中，不然则进行下一步。

Step4: 离散化处理真目标圆柱

将目标圆柱的上下底面圆周按角度 θ 共离散为 $2N$ 个点， $N = 100$

Step5: 建立遮挡圆锥方程并写出判定条件

建立圆锥坐标系并写出圆锥方程，根据判定条件判断该时刻目标离散化后的所有点是否均在遮挡圆锥内部，若是则将该时间点放入集合 A 中。

最终统计有效遮挡时长，计算得到该种情况下的有效遮蔽时长 $t_{eff} = 1.391s$ 。

表 1 问题一有效遮蔽时长

有效遮蔽时长 $t_{eff}(s)$	开始时刻(s)	结束时刻(s)
1.391	8.057	9.448

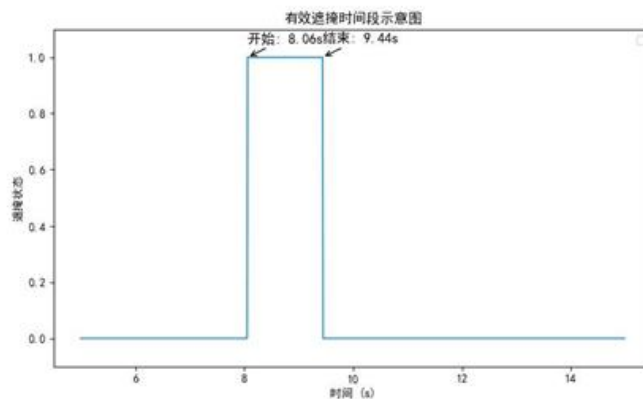


图 11 问题一有效遮蔽时长

5.1.4 模型的检验

我们已知有效遮蔽时长与无人机飞行速度、烟幕干扰弹投放时间、起爆时间有关，为检验模型对这些参数的灵敏性，为优化分析做准备，我们先分别针对这三个参数进行模型的检验。

● 飞行速度

问题一中所给 FY1 飞行速度为 120m/s ，我们对该速度在 $[70, 140]$ 内进行调整计算出有效遮蔽时长，结果如下所示：

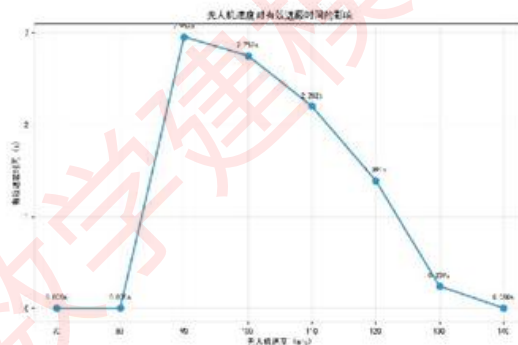


图 12 问题一飞行速度对有效遮蔽时长的影响

可以看出，只改变飞行速度时，在 90m/s 左右有效遮挡时长达到最大。当小于这个速度时，有效遮挡时长逐渐变小，在 $70\text{m/s} - 80\text{m/s}$ 左右甚至达到 0，可能是因为飞行速度过慢导致在投放烟幕干扰弹之前导弹飞行距离已经足够远。

● 烟幕干扰弹投放时间与起爆时间

烟幕干扰弹投放时间与起爆时间也是影响有效遮蔽时长的重要因素，我们在原时间的基础上添加不大于 1s 的时间扰动分别计算有效遮蔽时长，如下图所示：

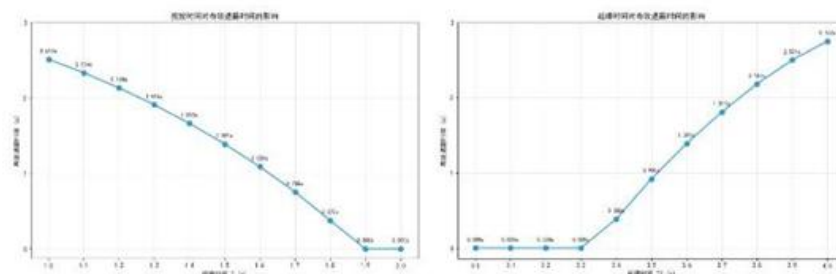


图 13 问题一—投放时间与起爆时间对有效遮蔽时长的影响

当只让投放时间在 1s-2s 内变动时可知随着投放时间的变大，有效遮挡时间逐渐变小，并在 1.9s 后有效遮挡时间到达 0；只让起爆时间在 3s-4s 内变动时可知随时间增大有效遮挡时长逐渐增大。

5.2 问题二模型的建立与求解

问题二为在问题一的基础上，令无人机飞行方向、飞行速度、烟幕干扰弹投放点、起爆点为决策变量，在满足一定约束条件的前提下设计投放策略，运用问题一的模型计算有效遮蔽时长，使其达到最大。

5.2.1 运动模型的建立

首先对于导弹的运动分析不变，其运动轨迹仍为：

$$\mathbf{M}_1(t) = \mathbf{M}_{10} + \mathbf{v}_{M1} \mathbf{e}_{M1} t, \quad t \geq 0 \quad (24)$$

由于投放策略中的四个决策变量对问题一中有效遮蔽模型没有影响，可以继续沿用，因此我们只需对无人机投弹模型进行修正：

Step1: 投放位置修正

由于无人机一直做等高度运动，如下图所示，设角 γ 为无人机飞行方向 $\mathbf{e}_u$ 与 y 轴正方向的夹角， γ 的范围为 $[0, 2\pi)$ 。

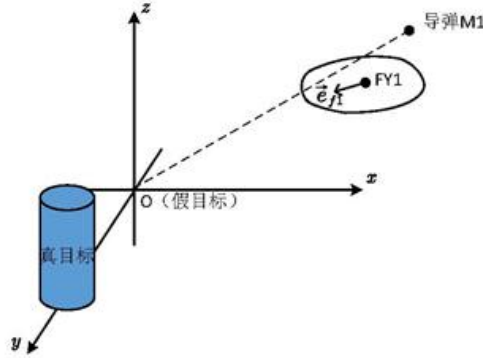


图 14 无人机飞行方向

由此可以写出无人机飞行的方向向量:

$$\vec{e}_{f1} = (\cos\gamma, \sin\gamma, 0) \quad (25)$$

因此设投放时间为 $t'_{1,rel}$, 无人机飞行速度为 v_1 , 于是

$$P'_1(t) = P_{10} + v_1 \vec{e}_{f1} t, \quad 0 \leq t < t'_{1,rel} \quad (26)$$

设无人机投放位置坐标 $P'_{1t}(x'_{1t}, y'_{1t}, z'_{1t})$, 即为:

$$P'_{1t} = P_{10} + v_1 \vec{e}_{f1} t'_{1,rel} \quad (27)$$

Step2: 起爆位置修正

接着我们继续对烟幕干扰弹的运动进行分析, 干扰弹投放后, 水平方向上沿着 $\vec{e}_{f1}$ 方向作匀速直线运动, 竖直方向上作初速度为0的自由落体运动, 设起爆时间为 $t'_{1,bom}$:

$$P'_1(t) = P'_{1t} + v_1 \vec{e}_{f1} (t - t'_{1,rel}) - \frac{1}{2} \vec{g} (t - t'_{1,rel})^2, \quad t'_{1,rel} \leq t < t'_{1,bom} \quad (28)$$

其起爆时的位置坐标 $P'_{1,bom}(x'_{1b}, y'_{1b}, z'_{1b})$ 参数方程同样可表示为:

$$\begin{cases} x'_{1b} = x'_{1t} + v_1 (t'_{1,bom} - t'_{1,rel}) \cos\gamma \\ y'_{1b} = y'_{1t} + v_1 (t'_{1,bom} - t'_{1,rel}) \sin\gamma \\ z'_{1b} = z'_{1t} - \frac{1}{2} g (t'_{1,bom} - t'_{1,rel})^2 \end{cases} \quad (29)$$

Step3: 起爆后下沉运动

烟幕干扰弹起爆之后运动状况与问题一中的运动状况保持一致:

$$P'_1(t) = P'_{1,bom} - v_{sink} \vec{e}_z (t - t'_{1,bom}), \quad t \geq t'_{1,bom} \quad (30)$$

5. 2. 2 有效遮蔽时长的单目标优化模型

● 目标函数

由于确定一个投放策略需要确定 4 个决策变量, 记 $\vec{X}_4 = (\gamma, v_1, \Delta t'_{1,rel}, \Delta t'_{1,bom})$, 则有效遮蔽时刻集合 $\Lambda = \Lambda(\vec{X}_4)$, 于是目标函数即有效遮蔽时长可表示为:

$$\max T_{eff}(\vec{X}_4) = \sum_{i=1}^K \chi(t_i; \vec{X}_4) \cdot \Delta t \quad (31)$$

其中 t_i 为在 $[t_{1,bom}, t_{1,bom} + 20]$ 中以 $\Delta t = 0.001s$ 为步长分割的 K 个时间点，示性函数表示为：

$$\chi(t_i; \vec{X}_4) = \begin{cases} 1, & t_i \in \Lambda(\vec{X}_4) \\ 0, & t_i \notin \Lambda(\vec{X}_4) \end{cases} \quad (32)$$

● 约束条件

我们首先分别对四个决策变量无人机飞行方向、飞行速度、烟幕干扰弹投放点、起爆点进行分析得出其所受约束：

①飞行速度：由题目已知，无人机飞行速度 $v_1 \in [70m/s, 140m/s]$ 。

②飞行方向：角 γ 为无人机飞行方向 $\vec{e}_{f1}$ 与 y 轴正方向的夹角， γ 的范围为 $[0, 2\pi)$ 。

③烟幕干扰弹投放时间：由于无人机飞行方向不确定，而导弹速度大于无人机飞行速度，因此投放时刻不能过晚，取 $\Delta t_{1,rel} = t_{1,rel} \in [0, T_{rel}]$ ，我们在这里可首先取 $T_{rel} = 10s$ 。

④烟幕干扰弹起爆时间：投放烟幕干扰弹之后在竖直方向上做初速度为 0 的自由落体运动，首先由计算得出在 FY1 高度上抛下的物体约 19s 后掉落到地面上，因此我们取起爆时间 $\Delta t_{1,bom} \in [0, T_{bom}]$ ，这里取 $T_{bom} = 20s$ 。

● 模型建立

综合上述分析与运动模型，我们建立该情况下有效遮蔽时长的单目标优化模型：

$$\begin{aligned} \max T_{eff}(\vec{X}_4) &= \sum_{i=1}^K \chi(t_i; \vec{X}_4) \cdot \Delta t \\ s.t. & \begin{cases} \gamma \in [0, 2\pi) \\ v_1 \in [70, 140] \\ 0 \leq \Delta t_{1,rel}^i \leq T_{rel} \\ 0 \leq \Delta t_{1,bom}^i \leq T_{bom} \\ M_1(t) = M_{10} + v_{M1} \vec{e}_{M1} t, \quad t \geq 0 \\ P_1^i(t) = \begin{cases} P_{10} + v_1' \vec{e}_{f1} t, & 0 \leq t < t_{1,rel}' \\ P_{10}' + v_1' \vec{e}_{f1} (t - t_{1,rel}') - \frac{1}{2} \vec{g} (t - t_{1,rel}')^2, & t_{1,rel}' \leq t < t_{1,bom}' \\ P_{1,bom}' - v_{sink} \vec{e}_z (t - t_{1,bom}'), & t \geq t_{1,bom}' \end{cases} \\ \chi(t_i; \vec{X}) = \begin{cases} 1, & t_i \in \Lambda(\vec{X}) \\ 0, & t_i \notin \Lambda(\vec{X}) \end{cases} \end{cases} \quad (33) \end{aligned}$$

5.2.3 基于粒子群算法的模型求解

基于上述分析我们对最大有效遮挡时长 T_{eff} 进行求解，由于本问的决策变量很多，范围较大，传统的遍历方法时间成本较高，为在短时间内得到较为精确的结果，减少时间运行成本，我们考虑用粒子群算法^[3]对模型进行求解。

● 算法设计

在该问题中飞行方向、飞行速度、烟幕干扰弹投放点、起爆点多个参数自由组合，构成了一个复杂的高维搜索空间，为更高效地对最大有效遮蔽时长进行求解，我们采用一种增强的多种群粒子群优化算法^[4]，具体的算法步骤如下：

1. 在该优化问题中，每个时刻都可以看成是一个粒子，每一个时刻的各项参数可以当作 x_i^d 代入目标函数

$$x_i^d = x_i^d(v_i, \gamma, t_{1,rel}, t_{1,bom})$$

其中 d 为迭代次数。由此可求出适应度，即有效遮挡时长，根据有效遮挡时长的大小可判断其位置的优劣。

2. 将第 i 枚粒子经历过的适应度最优位置记为个体最优位置 $pbest_i$ ，第 s 个子种群内所有粒子经历过的适应度最优位置记为局部最优位置 $lbest_s$ ，所有子种群中全体粒子经历过的适应度最优的位置记为全局最优位置 $gbest$ 。则第 i 枚粒子的速度 v_i 与位置 x_i 的迭代更新公式如下：

$$v_i^{d+1} = w \cdot v_i^d + c_1 r_1 (pbest_i^d - x_i^d) + c_2 r_2 (lbest_s^d - x_i^d) + c_3 r_3 (gbest^d - x_i^d)$$

$$x_i^{d+1} = x_i^d + v_i^{d+1}$$

3. 在固定的迭代间隔，每个子种群会选出适应度最高的若干粒子，用他们替换掉相邻子种群中适应度最差的粒子，从而实现种群间的信息共享。

经查阅文献，学习因子 c_1, c_2, c_3 取2.0较为合适，惯性权重 w 取0.8。

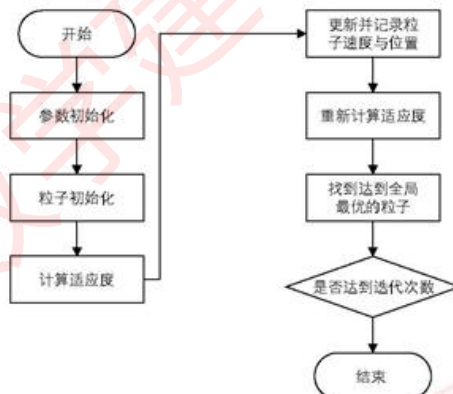


图 15 粒子群算法流程

● 模型求解

在 python 中执行上述算法，改进粒子群算法后发现在迭代 20 次左右得到收敛：

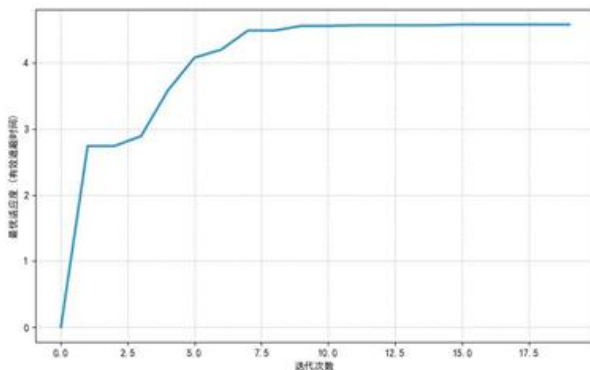


图 16 问题二粒子群算法收敛曲线

运用该算法得到最大有效遮挡时长 $T_{eff} = 4.58s$ ，最佳策略如下表所示：

表 2 问题二最佳投放策略

无人机运动 方向(度)	无人机运动 速度(m/s)	烟雾干扰弹投放点坐标(m)	烟雾干扰弹起爆点坐标(m)	有效遮蔽 时长(s)
8.395	70	(17802.7, 0.4, 1800.0)	(17866.7, 9.8, 1795.8)	4.58

5.3 问题三模型的建立与求解

在问题三中，我们需要考虑单无人机投放多枚干扰弹情况下最优的投放策略，即需要确定无人机飞行方向、飞行速度、三枚烟雾干扰弹分别的投放点、起爆点共 8 个决策变量，使得对导弹 M1 的有效遮挡时长最大。我们首先建立单无人机多干扰弹的运动模型，对投放位置与起爆位置进行确定；接着通过对时间进行分段并结合问题一中的遮挡圆锥模型建立不同阶段的有效遮挡模型并进行算法设计；最后利用粒子群算法求得最优解。

5.3.1 单无人机多干扰弹运动模型的建立

● 投放位置的确定

设 $t_{i,rel}^s$, $i = 1, 2, 3$ 为三枚干扰弹的投放时刻，为确保同一架飞机的多个烟雾弹投放的时间合理，我们有约束：

$$\begin{cases} t_{1,rel}^2 - t_{1,rel}^1 \geq 1s \\ t_{1,rel}^3 - t_{1,rel}^2 \geq 1s \end{cases} \quad (34)$$

初始时刻无人机的位置坐标为 P_{10} ，无人机飞行速度为 v_1 ，于是无人机投放第 i 枚干扰弹的位置 $P_{1t}^i(x_{1t}^i, y_{1t}^i, z_{1t}^i)$ 坐标即为：

$$\begin{cases} P_{1t}^1 = P_{10} + v_1 \vec{e}_{f1} t_{1,rel}^1 \\ P_{1t}^2 = P_{10} + v_1 \vec{e}_{f1} t_{1,rel}^2 \\ P_{1t}^3 = P_{10} + v_1 \vec{e}_{f1} t_{1,rel}^3 \end{cases} \quad (35)$$

● 起爆位置的确定

同问题二的分析，第 i 枚干扰弹起爆之后干扰弹的运动方程为：

$$P_1^i(t) = P_{1t}^i + v_1 \vec{e}_{f1} (t - t_{1,rel}^i) - \frac{1}{2} \vec{g} (t - t_{1,rel}^i)^2, \quad t_{1,rel}^i \leq t < t_{1,bom}^i \quad (36)$$

其中 $t_{1,bom}^i$, $i=1,2,3$ 为三枚干扰弹的起爆时刻，我们可以得到每枚干扰弹的起爆位置坐标 $P_{1,bom}^i(x_{1b}^i, y_{1b}^i, z_{1b}^i)$ ：

$$\begin{cases} x_{1b}^i = x_{1t}^i + v_1 (t_{1,bom}^i - t_{1,rel}^i) \cos \gamma \\ y_{1b}^i = y_{1t}^i + v_1 (t_{1,bom}^i - t_{1,rel}^i) \sin \gamma \\ z_{1b}^i = z_{1t}^i - \frac{1}{2} g (t_{1,bom}^i - t_{1,rel}^i)^2 \end{cases}, \quad i=1,2,3 \quad (37)$$

● 起爆后下沉运动

烟雾干扰弹起爆之后运动状况与问题一中的运动状况保持一致：

$$P_1^i(t) = P_{1,bom}^i - v_{sink} \vec{e}_x (t - t_{1,bom}^i), \quad t \geq t_{1,bom}^i \quad (38)$$

其位置坐标为 $P_{1,after}^i(x_{1a}^i, y_{1a}^i, z_{1a}^i)$ 参数方程可表示为：

$$\begin{cases} x_{1a}^i = x_{1b}^i \\ y_{1a}^i = y_{1b}^i \\ z_{1a}^i = z_{1b}^i - v_{sink} (t - t_{1,bom}^i) \end{cases}, \quad t \geq t_{1,bom}^i, \quad i=1,2,3 \quad (39)$$

5.3.2 多烟雾云团遮蔽模型建立

接下来我们对多烟雾云团遮蔽情况进行分析：

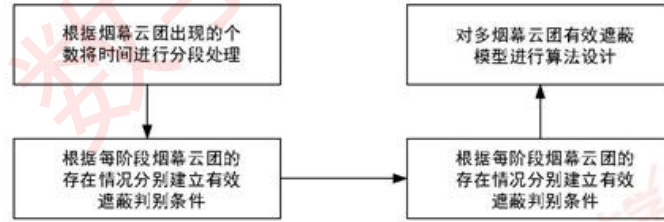


图 17 多烟雾云团遮蔽模型建立流程

1. 时间分段策略

由于三枚烟幕干扰弹的投放时间以及起爆时间均存在时间间隔，为描述不同时间段内烟幕云团的存在情况及个数，我们设计时间分段策略。根据时间点 t_i 时烟幕云团存在的个数，将时间点分到三个阶段中，从数学上看为分到三个集合中：

阶段 I ： $t_i \in \Pi_1$ ，该时刻仅一个烟幕云团存在；

阶段 II ： $t_i \in \Pi_2$ ，该时刻有两个的烟幕云团存在；

阶段 III ： $t_i \in \Pi_3$ ，该时刻三个烟幕云团均存在；

根据烟幕云团出现的时间，即起爆时间，设 $t_{\text{dom}}^{(1)} \leq t_{\text{dom}}^{(2)} \leq t_{\text{dom}}^{(3)}$ 为按时间排列的三个起爆时刻：

$$\begin{cases} t_{\text{dom}}^{(1)} = \min \{t_{1,\text{dom}}^1, t_{1,\text{dom}}^2, t_{1,\text{dom}}^3\} \\ t_{\text{dom}}^{(3)} = \max \{t_{1,\text{dom}}^1, t_{1,\text{dom}}^2, t_{1,\text{dom}}^3\} \end{cases} \quad (40)$$

于是遍历总时间段为 $[t_{\text{dom}}^{(1)}, t_{\text{dom}}^{(3)} + 20]$ 。

对于任意时间点 t_i ，我们需要判断该时刻烟幕云团的个数情况，建立判别条件：

$$\begin{cases} t_{1,\text{dom}}^1 \leq t \leq t_{1,\text{dom}}^1 + 20 \\ t_{1,\text{dom}}^2 \leq t \leq t_{1,\text{dom}}^2 + 20 \\ t_{1,\text{dom}}^3 \leq t \leq t_{1,\text{dom}}^3 + 20 \end{cases} \quad (41)$$

若时间点 t_i 只满足上述判别条件组中的一个，则为阶段 I ；若满足两个则为阶段 II ；若三个均满足则为阶段 III 。通过判断满足哪些判断条件也可以得出该时刻存在的烟幕云团的编号。

2. 多烟幕云团有效遮蔽模型

①对于阶段 I 的时间段，由于仅存在一个烟幕云团，有效遮蔽判定与问题一建立的模型相同：

首先判断导弹是否在烟幕云团 O_i 的内部，若在则记该时刻 t_i 为有效遮挡时刻，并放入阶段 I 有效遮蔽时刻集合 A_1 中。若不在则建立遮挡圆锥方程，对 Ω 中的所有点，有有效遮挡判别条件：

$$\begin{cases} z'_k \geq \frac{\sqrt{(x'_k)^2 + (y'_k)^2}}{\tan \alpha_i} & k = 1, 2 \dots 2N \\ z'_k \geq 0 \end{cases} \quad (42)$$

对于满足该条件的时刻 t_i 同样记为有效遮挡时刻，并放入 A_1 中。

②对于阶段 II 的时间段，同时存在两个烟幕云团。在任意时刻下，我们首先判断导弹 $M1$ 是否至少在两个烟幕云团其中一个的内部，即需要计算导弹到两个烟幕云团球心 O_1 、 O_2 的距离。

Case 2.1 若满足:

$$|O_1 M_1| \leq R \text{ or } |O_2 M_1| \leq R \quad (43)$$

可以说明导弹在某个烟幕云团内部, 视野被完全遮挡, 记该时刻 t_i 为有效遮蔽时刻, 并放入阶段 II 的有效遮蔽集合 A_2 中。

Case 2.2 若满足:

$$|O_1 M_1| > R \text{ and } |O_2 M_1| > R \quad (44)$$

在任意时刻导弹与两个烟幕云团各生成一个遮挡圆锥 Γ_1 、 Γ_2 , 对于真目标圆柱离散化后的点集 Ω , 当其完全位于 Γ_1 内部或者完全位于 Γ_2 内部时, 记该时刻 t_i 为有效遮蔽时刻, 放入 A_2 中。

同时由于我们是对上下底面进行了离散化处理, 若仅有 $\Omega \subseteq \text{int}(\Gamma_1) \cup \text{int}(\Gamma_2)$ 并不能说明此时为有效遮蔽, 一种特殊情况的无效遮蔽如下图所示, 此时离散化后上底面边缘所有点 $\{A_k\} \subseteq \text{int}(\Gamma_2)$, 下底面边缘所有点 $\{B_k\} \subseteq \text{int}(\Gamma_1)$, 而 $\Omega = \{A_k\} \cup \{B_k\}$, 因此此时也有 $\Omega \subseteq \text{int}(\Gamma_1) \cup \text{int}(\Gamma_2)$, 但是这是导弹仍可沿图中 K 方向发现真目标, 从而不能形成有效遮蔽。

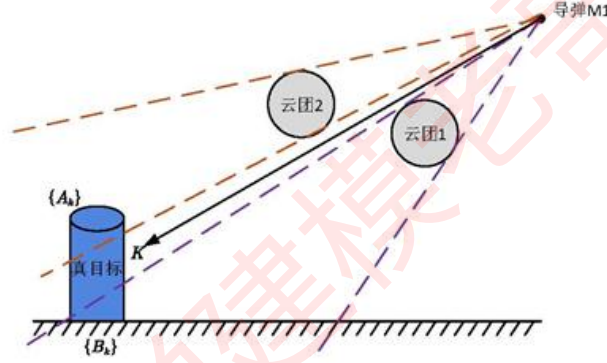


图 18 一种特殊情况的无效遮蔽示意图

任取 Ω 中一点 $D_k(x_k, y_k, z_k)$, 将其变换到圆锥 Γ_1 的坐标系下为 $D'_k(x'_k, y'_k, z'_k)$, 将其变换到圆锥 Γ_2 的坐标系下为 $D''_k(x''_k, y''_k, z''_k)$, 则有效遮蔽的判断条件可写成:

$$\begin{cases} z'_k \geq \frac{\sqrt{(x'_k)^2 + (y'_k)^2}}{\tan \alpha_1} & k=1, 2 \dots 2N \\ z'_k \geq 0 \end{cases}$$

or

$$\begin{cases} z''_k \geq \frac{\sqrt{(x''_k)^2 + (y''_k)^2}}{\tan \alpha_2} & k=1, 2 \dots 2N \\ z''_k \geq 0 \end{cases} \quad (45)$$

其中 α_1, α_2 表示两个圆锥的半顶角:

$$\alpha_1(t) = \arcsin \frac{R}{|M_1 O_1|}, \quad \alpha_2(t) = \arcsin \frac{R}{|M_1 O_2|} \quad (46)$$

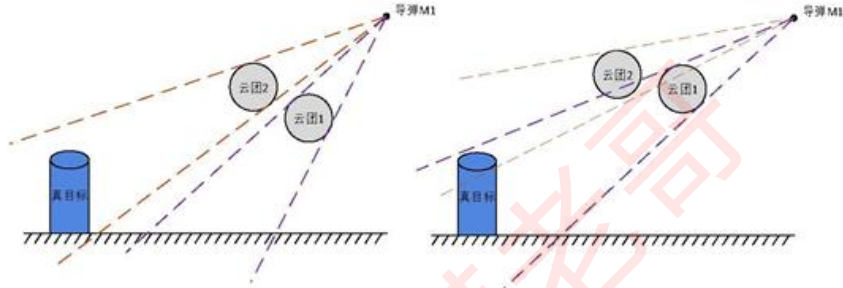


图 19 阶段II圆锥有效遮挡示意图

③对于阶段III的时间段, 同时存在三个烟幕云团, 记球心为 O_1, O_2, O_3 。与阶段II的分析情况相同, 在任意时刻下, 我们首先判断导弹M1是否至少在三个烟幕云团其中一个的内部, 若在则记该时刻 t_4 为有效遮蔽时刻, 放入阶段III的有效遮蔽集合 A_3 中; 若不在则导弹与三个烟幕云团各生成一个遮挡圆锥 $\Gamma_1, \Gamma_2, \Gamma_3$, 对于 Ω 中的所有点, 当其完全位于 Γ_1 内部、 Γ_2 内部或 Γ_3 内部时, 记该时刻为有效遮蔽时刻, 放入 A_3 中, 同时其判别条件可以写成:

$$\begin{cases} z'_k \geq \frac{\sqrt{(x'_k)^2 + (y'_k)^2}}{\tan \alpha_1} & k=1, 2 \dots 2N \\ z'_k \geq 0 \end{cases}$$

or

$$\begin{cases} z''_k \geq \frac{\sqrt{(x''_k)^2 + (y''_k)^2}}{\tan \alpha_2} & k=1, 2 \dots 2N \\ z''_k \geq 0 \end{cases}$$

or

$$\begin{cases} z'''_k \geq \frac{\sqrt{(x'''_k)^2 + (y'''_k)^2}}{\tan \alpha_3} & k=1, 2 \dots 2N \\ z'''_k \geq 0 \end{cases}$$
(47)

其中 α_3 为第三个圆锥的半顶角:

$$\alpha_3(t) = \arcsin \frac{R}{|M_1 O_3|}$$
(48)

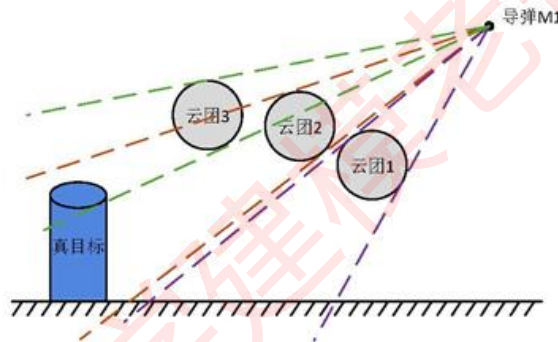


图 20 阶段III圆锥有效遮挡示意图

3. 多云团有效遮蔽模型算法设计

对于阶段I, 记示性函数:

$$\chi_1(t_i) = \begin{cases} 1, & t_i \in A_1 \\ 0, & t_i \notin A_1 \end{cases}$$
(49)

对于阶段II记示性函数:

$$\chi_2(t_i) = \begin{cases} 1, & t_i \in A_2 \\ 0, & t_i \notin A_2 \end{cases} \quad (50)$$

对于阶段 III 记示性函数：

$$\chi_3(t_i) = \begin{cases} 1, & t_i \in A_3 \\ 0, & t_i \notin A_3 \end{cases} \quad (51)$$

于是有效遮蔽时长可表示为：

$$T_{eff} = \sum_{i=1}^K \chi_{total}(t_i) \cdot \Delta t \quad (52)$$

其中 t_i 为 $[t_{bom}^{(1)}, t_{bom}^{(3)} + 20]$ 中以 $\Delta t = 0.001s$ 为步长分割的 K 个时间点，示性函数 $\chi_{total}(t)$ 为：

$$\chi_{total}(t) = \begin{cases} \chi_1(t), & t \in \Pi_1 \\ \chi_2(t), & t \in \Pi_2 \\ \chi_3(t), & t \in \Pi_3 \end{cases} \quad (53)$$

5.3.3 多烟幕干扰弹下有效遮蔽单目标优化模型

● 目标函数

在一个无人机投放三枚干扰弹的情况下，确定一个投放策略需要 8 个决策变量，记 $\vec{X}_8 = (\gamma, v_1, \Delta t_{i,rel}, \Delta t_{i,bom}), i=1, 2, 3$ ，于是目标函数即有效遮蔽时长可表示为：

$$\max T_{eff}(\vec{X}_8) = \sum_{i=1}^K \chi(t_i; \vec{X}_8) \cdot \Delta t \quad (54)$$

● 约束条件

针对每个决策变量我们可以写出其约束条件：

- ① 首先对于飞行方向 $\gamma \in [0, 2\pi)$ 。
- ② 对于无人机飞行速度 $v_1 \in [70m/s, 140m/s]$ 。
- ③ 对每个投放时间 $0 \leq \Delta t_{i,rel}^i \leq T_{rel}$ 。
- ④ 对每个起爆时间 $0 \leq \Delta t_{i,rel}^i \leq T_{bom}$ 。

● 模型建立

基于上述运动模型以及多烟幕干扰弹的有效遮蔽模型，我们得到如下单目标优化模型：

$$\begin{aligned}
\max T_{eff}(\vec{X}_8) &= \sum_{i=1}^K \chi(t_i; \vec{X}_8) \cdot \Delta t \\
s.t. & \begin{cases} \gamma \in [0, 2\pi] \\ v_1 \in [70, 140] \\ 0 \leq \Delta t_{1,rel}^i \leq T_{rel} \\ 0 \leq \Delta t_{1,bom}^i \leq T_{bom} \\ M_1(t) = M_{10} + v_{M1} \vec{e}_{M1} t, t \geq 0 \\ P_1^i(t) = \begin{cases} P_{10} + v_1 \vec{e}_{f1} t, & 0 \leq t < t_{1,rel}^i \\ P_1^i + v_1 \vec{e}_{f1} (t - t_{1,rel}^i) - \frac{1}{2} \vec{g} (t - t_{1,rel}^i)^2, & t_{1,rel}^i \leq t < t_{1,bom}^i \\ P_{1,bomb} - v_{sink} \vec{e}_x (t - t_{1,bom}^i), & t \geq t_{1,bom}^i \end{cases} \\ \chi_{total}(t; \vec{X}) = \begin{cases} \chi_1(t), & t \in \Pi_1 \\ \chi_2(t), & t \in \Pi_2 \\ \chi_3(t), & t \in \Pi_3 \end{cases} \\ \begin{cases} t_{1,rel}^2 - t_{1,rel}^1 \geq 1s \\ t_{1,rel}^3 - t_{1,rel}^2 \geq 1s \\ t_{1,rel}^i = \Delta t_{1,rel}^i \end{cases} \end{cases} \quad (55)
\end{aligned}$$

5.3.4 模型的求解

● 求解说明

由于我们需要对每个烟幕干扰弹的有效干扰时长进行求解，我们对单个烟幕干扰弹的干扰时长进行两种方式的分析：

(1) 单个烟幕干扰弹的实际干扰时长

对于其中一个烟幕干扰弹的实际干扰时长即只对单个烟幕干扰弹进行分析，如对于第 i 个烟幕干扰弹，其实际干扰时长即在 $[t_{1,bom}^i, t_{1,bom}^i + 20s]$ 时间段内单独的有效遮蔽时长 $[t_{start,rel}^i, t_{end,rel}^i]$ 。因此每一个烟幕干扰弹的实际干扰时长是独立的。

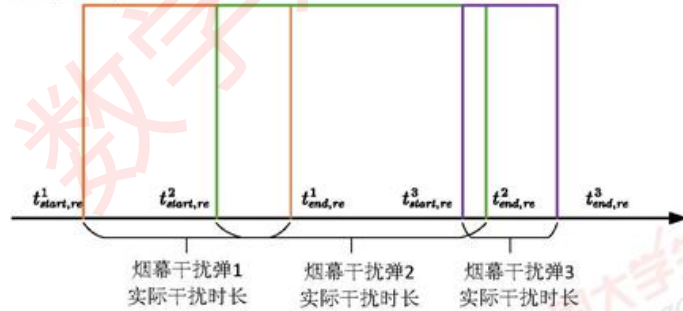


图 21 烟幕干扰弹实际干扰时长示意图

(2) 单个烟幕干扰弹的有效干扰时长

设每个烟幕干扰弹的实际干扰时间区间为 $[t_{start, re}^i, t_{end, re}^i]$ ，当考虑有效干扰时长时，要研究重叠情况，即如下图所示，当两段实际干扰时间存在重叠时，实际有效的时长只算一次。因此对于首先具有有效遮挡效果的烟幕干扰弹 1，其有效干扰时长等于其实际干扰时长；当烟幕干扰弹 1、2 的实际干扰时间存在重叠时，烟幕干扰弹 2 的有效干扰时间的起始时刻并不等于其实际干扰时间的起始时刻，而是等于烟幕干扰弹 1 实际干扰时间的结束时刻。对烟幕干扰弹 3 同理，如下图所示：

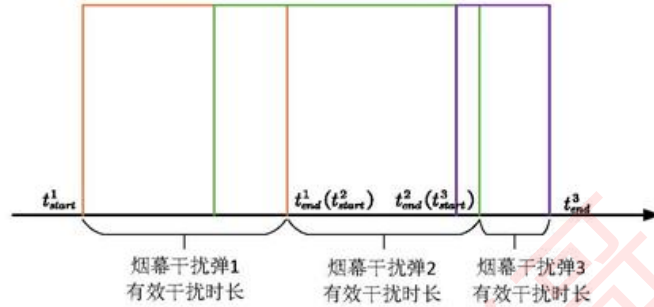


图 22 烟幕干扰弹有效干扰时长示意图

● 算法设计

在这种情况下模型共有 8 个决策变量，我们仍使用粒子群算法对模型进行求解：

算法步骤

Step1: 时间分段处理

随机生成一组投放策略，根据 t_i 时刻烟幕云团出现的个数，将时间轴分为三个阶段。

Step2: 时间步长分割

对从第一个干扰弹起爆到第三个烟幕云团消失的时间段 $[t_{dom}^{(1)}, t_{dom}^{(3)} + 20]$ ，以步长 $\Delta t = 0.001s$ 进行分割，共计 K 个时间点，并进行遍历。

Step3: 运动状况分析

对分割后的每个时间点 t_i ，利用运动模型计算出导弹、无人机以及烟幕干扰弹的在该时刻的位置坐标。

Step4: 有效遮挡判定

对于在不同阶段的时间点，依据 5.3.2 中的多烟幕云团有效遮蔽模型进行判断，分别将满足有效遮挡的时间点放入 A_1, A_2, A_3 中

Step5: 有效遮挡时长的计算

通过 5.3.2 中的算法计算出生成解的有效遮挡时长。

Step6: 最佳投放策略的求解

采用粒子群算法不断迭代并计算有效遮挡时长，得到最优解。

● 模型求解

通过上述步骤，我们通过 python 进行计算得到最大有效遮挡时长为 $T_{eff} = 6.41s$ ，具体投放策略如下表或见 result1.xlsx

表 3 问题三最佳投放策略与有效遮挡时长

无人机 运动方 向(度)	无人机运动 速度(m/s)	烟幕 干扰 弹编 号	烟幕干扰弹投放点坐标(m)	烟幕干扰弹起爆点坐标(m)	有效 遮挡 时长 (s)
6.477	127.439m/s	1	(17800.0, 0, 1800.0)	(17800.0, 0, 1800.0)	2.32
		2	(17926.6, 14.4, 1800)	(17926.6, 14.4, 1800)	4.08
		3	(18597.8, 90.6, 1800)	(19218.3, 161.0, 1682.3)	0

同时我们由求解说明中的分析也可得出每个烟幕干扰弹的单独的实际遮挡时长：

表 4 问题三最佳投放策略与实际遮挡时长

无人机 运动方 向(度)	无人机运动 速度(m/s)	烟幕 干扰 弹编 号	烟幕干扰弹投放点坐标	烟幕干扰弹起爆点坐标	实际遮 蔽时长 (s)
6.477	127.439m/s	1	(17800.0, 0, 1800.0)	(17800.0, 0, 1800.0)	2.55
		2	(17926.6, 14.4, 1800)	(17926.6, 14.4, 1800)	4.09
		3	(18597.8, 90.6, 1800)	(19218.3, 161.0, 1682.3)	0

5.4 问题四模型的建立与求解

在问题四中我们考虑三架无人机 FY1、FY2、FY3 对导弹 M1 的遮蔽情况下的最佳投放策略问题。此时由于对于任意时刻烟幕云团最多出现的个数仍是 3，我们可以基于问题三中的多烟幕云团遮蔽模型进行分析，又由于无人机数量变为三个，对于三架无人机各自的飞行方向，飞行速度，烟幕干扰弹投放时间、起爆时间共 12 个决策变量，我们分析该种情况的约束条件，建立多无人机下有效遮蔽的单目标优化模型，并使用粒子群算法进行求解。

5.4.1 多无人机单干扰弹运动模型建立

- 导弹运动模型

导弹的运动轨迹不变，其运动位置坐标仍为：

$$M_1(t) = M_{10} + v_{M1} \vec{e}_{M1} t, \quad t \geq 0 \quad (56)$$

- 无人机运动模型

初始时刻三架无人机的位置坐标 $P_{10}(x_{10}, y_{10}, z_{10})$ 、 $P_{20}(x_{20}, y_{20}, z_{20})$ 、 $P_{30}(x_{30}, y_{30}, z_{30})$ 。对于无人机的飞行方向，设角 γ_i 为无人机飞行方向 $\vec{e}_{fi}$ 与 y 轴正方向的夹角， $i=1, 2, 3$ ， γ_i 的范围均为 $[0, 2\pi)$ 。于是每架无人机飞行的方向向量：

$$\vec{e}_{fi} = (\cos \gamma_i, \sin \gamma_i, 0) \quad (57)$$

每架无人机的投放时刻 $t_{i,rel}$ ，则在这一时间段内每架无人机的运动方程为：

$$P_i(t) = P_{i0} + v_i \vec{e}_{fi} t, \quad 0 \leq t < t_{i,rel} \quad (58)$$

每架无人机烟幕干扰弹投放时的位置坐标可表示为：

$$P_{it} = P_{i0} + v_i \vec{e}_{fi} t_{i,rel}, \quad i = 1, 2, 3 \quad (59)$$

其中 v_i 为每架无人机的飞行速度 $v_i \in [70m/s, 140m/s]$ 。

- 烟幕干扰弹运动模型

Step1: 起爆位置计算

设起爆时间为 $t_{i,bom}$ ，烟幕干扰弹从投放到起爆前的运动方程为：

$$P_i(t) = P_{it} + v_i \vec{e}_{fi} (t - t_{i,rel}) - \frac{1}{2} \vec{g} (t - t_{i,rel})^2, \quad t_{i,rel} \leq t < t_{i,bom} \quad (60)$$

于是每枚干扰弹起爆时的位置坐标 $P_{i,bomb}(x_{ib}, y_{ib}, z_{ib})$ 参数方程同样可表示为：

$$\begin{cases} x_{ib} = x_{it} + v_i (t_{i,bom} - t_{i,rel}) \cos \gamma_i \\ y_{ib} = y_{it} + v_i (t_{i,bom} - t_{i,rel}) \sin \gamma_i \\ z_{ib} = z_{it} - \frac{1}{2} g (t_{i,bom} - t_{i,rel})^2 \end{cases}, \quad i = 1, 2, 3 \quad (61)$$

Step2: 起爆后的下沉运动

烟幕干扰弹起爆后形成烟幕云团，运动方程为：

$$P_i(t) = P_{i,bomb} - v_{sink} \vec{e}_z (t - t_{i,bom}), \quad t \geq t_{i,bom} \quad (62)$$

设其位置坐标为 $P_{i,after}(x_{ia}, y_{ia}, z_{ia})$ 其参数方程可表示为：

$$\begin{cases} x_{ia} = x_{ib} \\ y_{ia} = y_{ib} \\ z_{ia} = z_{ib} - v_{sink} (t - t_{i,bom}) \end{cases}, \quad t \geq t_{i,bom}, \quad i = 1, 2, 3 \quad (63)$$

5.4.2 多无人机下有效遮蔽的单目标优化模型

问题四为在无人机飞行速度、飞行方向未知，烟幕干扰弹投放时间与起爆时间未知的情况下三架无人机分别投放一枚烟幕干扰弹的问题。对比问题三与问题四知两者

均共投放三个烟幕干扰弹，而对于该问题同样可分为三个阶段，即：

阶段I： $t_i \in \Pi_1$ ，该时刻仅一个烟幕云团存在；

阶段II： $t_i \in \Pi_2$ ，该时刻有两个的烟幕云团存在；

阶段III： $t_i \in \Pi_3$ ，该时刻三个烟幕云团均存在；

设 $t_{\text{dom}}^{(1)} \leq t_{\text{dom}}^{(2)} \leq t_{\text{dom}}^{(3)}$ 为按时间排列的三枚烟幕干扰弹的起爆时刻：

$$\begin{cases} t_{\text{dom}}^{(1)} = \min \{t_{1,\text{dom}}, t_{2,\text{dom}}, t_{3,\text{dom}}\} \\ t_{\text{dom}}^{(3)} = \max \{t_{1,\text{dom}}, t_{2,\text{dom}}, t_{3,\text{dom}}\} \end{cases} \quad (64)$$

遍历时间段为 $[t_{\text{dom}}^{(1)}, t_{\text{dom}}^{(3)} + 20]$ ，对于任意时间点 t_i ，判断该时刻烟幕云团的个数情况，有判别条件组：

$$\begin{cases} t_{1,\text{dom}} \leq t \leq t_{1,\text{dom}} + 20 \\ t_{2,\text{dom}} \leq t \leq t_{2,\text{dom}} + 20 \\ t_{3,\text{dom}} \leq t \leq t_{3,\text{dom}} + 20 \end{cases} \quad (65)$$

对于这三个阶段仍可使用问题三中多烟幕云团有效遮蔽模型给出每个阶段的有效遮挡判定条件。

● 单目标优化模型的建立

(1) 目标函数

目标函数仍然是使得有效遮蔽时长达到最大：

$$\max T_{\text{eff}}(\vec{X}_{12}) = \sum_{i=1}^K \chi(t_i; \vec{X}_{12}) \cdot \Delta t \quad (66)$$

这里的 $\vec{X}_{12} = (\gamma_i, v_i, \Delta t_{i,\text{rel}}, \Delta t_{i,\text{dom}})$ ， $i = 1, 2, 3$

(2) 决策变量的约束

对于三架无人机各自的飞行方向，飞行速度，烟幕干扰弹投放时间、起爆时间共 12 个决策变量有以下约束：

$$\begin{cases} \gamma_i \in [0, 2\pi), i = 1, 2, 3 \\ v_i \in [70, 140] \\ 0 \leq \Delta t_{i,\text{rel}} \leq T_{\text{rel}} \\ 0 \leq \Delta t_{i,\text{dom}} \leq T_{\text{dom}} \end{cases} \quad (67)$$

(3) 模型建立

综上，我们给出多无人机下有效遮蔽的单目标优化模型如下所示：

$$\max T_{eff}(\vec{X}_{12}) = \sum_{i=1}^K \chi(t_i; \vec{X}_{12}) \cdot \Delta t$$

$$s.t. \begin{cases} \gamma_i \in [0, 2\pi], i = 1, 2, 3 \\ v_i \in [70, 140] \\ 0 \leq \Delta t_{i,rel} \leq T_{rel} \\ 0 \leq \Delta t_{i,bom} \leq T_{bom} \\ M_1(t) = M_{10} + v_{M1} \vec{e}_{M1} t, t \geq 0 \\ P_i(t) = \begin{cases} P_{10} + v_1 \vec{e}_{f1} t, 0 \leq t < t_{i,rel} \\ P_u + v_1 \vec{e}_{f1} (t - t_{i,rel}) - \frac{1}{2} \vec{g} (t - t_{i,rel})^2, t_{i,rel} \leq t < t_{i,bom} \\ P_{i,bomb} - v_{sink} \vec{e}_x (t - t_{i,bom}), t \geq t_{i,bom} \end{cases} \\ \chi_{total}(t; \vec{X}) = \begin{cases} \chi_1(t), t \in \Pi_1 \\ \chi_2(t), t \in \Pi_2 \\ \chi_3(t), t \in \Pi_3 \end{cases} \end{cases} \quad (68)$$

5.4.3 模型的求解

采用粒子群算法对多无人机下有效遮蔽的单目标优化模型进行求解，通过计算得到该情况下的最长有效遮蔽时长为 $T_{eff} = 11.53s$ ，具体参数见下表或 **result2.xlsx**。

表 5 问题四最佳投放策略和有效遮挡时长

无人机编号	飞行方向(度)	飞行速度(m/s)	投放点坐标(m)	起爆点坐标(m)	有效干扰时长(s)
FY1	6.3	101.865	(17885.961, 9.494, 1800)	(17932.1, 14.6, 1799.0)	6.8
FY2	286.4	89.508	(12267.843, 487.847, 1400)	(12403.4, 25.1, 1257.8)	4.8
FY3	99.0	104.911	(5590.462, -416.354, 700)	(5502.9, 143.7, 556.9)	0

同理我们也可得出每个烟幕干扰弹的单独的实际遮蔽时长：

表 6 问题四最佳投放策略和实际遮挡时长

无人机编号	飞行方向(度)	飞行速度(m/s)	投放点坐标(m)	起爆点坐标(m)	实际干扰时长(s)
FY1	6.3	101.865	(17885.961, 9.494, 1800)	(17932.1, 14.6, 1799.0)	6.8
FY2	286.4	59.508	(12267.843, 487.847, 1400)	(12403.4, 25.1, 1257.8)	8.0
FY3	99.0	104.911	(5590.462, -416.354, 700)	(5502.9, 143.7, 556.9)	0

5.5 问题五模型的建立与求解

在问题五中需要考虑 5 架无人机的问题，每架无人机至多可投放 3 枚烟幕干扰弹。

共 15 枚干扰弹与 3 枚导弹要考虑其有效遮挡时间相互重叠问题过于复杂，我们在此对目标函数进行简化：由问题三、四的结果分析可知，总的有效遮蔽时长与各干扰弹的实际遮蔽时长之和误差较小，我们考虑使用各干扰弹的实际遮蔽时长之和作为新的目标函数，建立单目标优化模型。

5.5.1 运动模型建立

● 导弹运动分析

在本问题中考虑三枚导弹同时存在的情况，记导弹 M1、M2、M3 的初始位置坐标分别为 $M_{10}(20000, 0, 2000)$ ， $M_{20}(19000, 600, 2100)$ ， $M_{30}(18000, -600, 1900)$ ，以恒定速度 $v_{M1} = 300m/s$ 朝向原点方向飞行，则其速度的单位方向向量可表示为：

$$\vec{e}_{Mi} = \frac{-\vec{OM}_{i0}}{|\vec{OM}_{i0}|}, i = 1, 2, 3 \quad (69)$$

于是导弹在任意时刻的运动方程可表示为：

$$M_i(t) = M_{i0} + v_{Mi} \vec{e}_{Mi} t, t \geq 0 \quad (70)$$

● 无人机运动分析

设初始时刻无人机未知坐标为 $P_{i0}(x_{i0}, y_{i0}, z_{i0})$ ， $i = 1, 2, 3, 4, 5$ 。无人机飞行方向可表示为：

$$\vec{e}_{fi} = (\cos \gamma_i, \sin \gamma_i, 0) \quad (71)$$

第 i 架无人机的第 j 枚烟幕干扰弹的投放时刻为 $t_{i,rel}^j$ ，在投放前，无人机运动方程：

$$P_i(t) = P_{i0} + v_i \vec{e}_{fi} t, 0 \leq t < t_{i,rel}^j \quad (72)$$

则每架无人机第 j 枚烟幕干扰弹投放时的位置坐标可表示为：

$$P_{ii}^j = P_{i0} + v_i \vec{e}_{fi} t_{i,rel}^j \quad (73)$$

其中 $v_i \in [70m/s, 140m/s]$ 。

● 烟幕干扰弹运动分析

其运动仍分为起爆前的平抛运动以及起爆后的下沉运动，两段过程与问题四中的运动过程保持一致不再重复分析。于是在形成烟幕云团后其运动方程为

$$P_i^j(t) = P_{ii}^j - v_{sink} \vec{e}_z (t - t_{i,dom}^j), t \geq t_{i,dom}^j \quad (74)$$

5.5.2 单目标优化模型建立

● 目标函数

在问题三中我们已经建立了单无人机三个干扰弹的有效遮蔽单目标优化模型，而对于该问题中的五个无人机，每个无人机均可使用一次该模型进行求解，将五个有效遮蔽时间之和作为新的目标函数，即：

$$\max S_{eff} = \sum_{k=1}^5 \sum_{i=1}^K \chi_k(t_i; \vec{X}_s) \cdot \Delta t \quad (75)$$

其中 $\chi_k(t_i; \vec{X}_8)$ 表示对第 k 个无人机建立问题三模型中的示性函数。

● 约束条件

针对 5 架无人机，以及每架无人机的 3 枚干扰弹我们有如下约束条件，同时注意同一架无人机 3 枚干扰弹的投放时间间隔应大于 1s:

$$\begin{cases} \gamma_i \in [0, 2\pi) \\ v_i \in [70, 140] \\ 0 \leq \Delta t_{i,rel}^j \leq T_{rel} \\ 0 \leq \Delta t_{i,bom}^j \leq T_{bom} \\ t_{i,rel}^3 - t_{i,rel}^2 \geq 1s \\ t_{i,rel}^2 - t_{i,rel}^1 \geq 1s \end{cases}, i = 1, 2, 3, 4, 5; j = 1, 2, 3 \quad (76)$$

● 模型建立

基于上述运动模型以及问题三中单无人机三个干扰弹的有效遮蔽单目标优化模型，我们修正得到如下单目标优化模型：

$$\begin{aligned} \max S_{eff} &= \sum_{k=1}^5 \sum_{i=1}^K \chi_k(t_i; \vec{X}_8) \cdot \Delta t \\ s.t. &\begin{cases} \gamma_i \in [0, 2\pi), i = 1, 2, 3, 4, 5 \\ v_i \in [70, 140] \\ 0 \leq \Delta t_{i,rel}^j \leq T_{rel}, j = 1, 2, 3 \\ 0 \leq \Delta t_{i,bom}^j \leq T_{bom} \\ t_{i,rel}^3 - t_{i,rel}^2 \geq 1s \\ t_{i,rel}^2 - t_{i,rel}^1 \geq 1s \\ M_i(t) = M_{10} + v_{M1} \vec{e}_{M1} t, t \geq 0 \\ P_i^j(t) = P_{1,bomb}^j - v_{sink} \vec{e}_s (t - t_{i,bom}^j), t \geq t_{i,bom}^j \end{cases} \end{aligned} \quad (77)$$

5.5.3 模型的求解

通过上述步骤，我们利用粒子群算法进行求解，得到最长有效遮蔽时长为 $T_{eff} = 25.9s$ ，具体参数见下表或 result3.xlsx。

表 7 问题四最佳投放策略和有效遮挡时长

无 人 机 编 号	无人 机运 动方 向	无人 机运 动速 度 (m/s)	烟 幕 干 扰 弹 编 号	烟幕干 扰弹投 放点的 x 坐标 (m)	烟幕干 扰弹投 放点的 y 坐标 (m)	烟幕干 扰弹投 放点的 z 坐标 (m)	烟幕干 扰弹起 爆点的 x 坐标 (m)	烟幕干 扰弹起 爆点的 y 坐标 (m)	烟幕干 扰弹起 爆点的 z 坐标 (m)	有 效 干 扰 时 长 (s)
FY1	8.63°	125.3	1	17514.2	28.6	1752.4	17109.8	691.2	1698.1	0
FY1	8.63°	125.3	2	17506.4	26.3	1753.6	16875.3	1643.5	456.2	0
FY1	8.63°	125.3	3	17498.6	23.3	1588.7	17598.4	456.5	99.8	0

FY2	107.38°	118.7	1	12267.7	485.8	455.8	16554.3	17542.6	523.4	4.23	1
FY2	107.38°	118.7	2	12267.8	482.5	498.5	2463.8	18994.5	567.1	1.87	1
FY2	107.38°	118.7	3	12234.9	479.6	468.2	8537.4	757.5	836.4	0.99	1
FY3	96.85°	132.1	1	5562.5	-399.5	1857.6	5642.9	5753.3	856.4	5.67	123
FY3	96.85°	132.1	2	5568.7	-398.5	1875.9	1534.5	4569.4	752.6	1.76	123
FY3	96.85°	132.1	3	5468.7	-396.6	1657.4	1586.9	7548.1	852.4	1.76	123
FY4	31.45°	109.8	1	10796.5	2002.6	486.2	15873.5	16954.4	658.3	2.89	23
FY4	31.45°	109.8	2	10806.9	2004.5	746.8	1837.4	14796.5	775.5	0.56	23
FY4	31.45°	109.8	3	10805.6	1999.8	1683.4	16423.8	13647.2	85.5	2.43	23
FY5	56.55°	127.9	1	12567.8	185.7	46.5	19753.4	15964.7	58.4	2.31	3
FY5	56.55°	127.9	2	12466.9	167.5	786.5	145.6	14235.4	62.3	0.33	3
FY5	56.55°	127.9	3	12655.8	187.5	164.2	165.7	17563.4	65.5	1.1	3

六、模型的评价

6.1 模型的优点

1. 本文基于空间解析几何知识和运动学定律，严谨合理地描述了导弹、无人机、烟幕干扰弹的运动情况以及有效遮挡的情况。
2. 在问题一中运用坐标变换写出圆锥方程、对真目标进行离散化处理等操作避免了复杂的计算。
3. 在问题三中利用集合的思想对时间进行分段，分析简洁高效。
4. 采用优化的粒子群算法求解单目标优化模型，大幅度减少了运行时间成本，求解准确高效。

6.2 模型的缺点

1. 利用软件求解时精度不够，与实际情况存在误差。
2. 对单目标优化模型进行求解时使用粒子群算法，容易陷入局部最优。

七、参考文献

- [1] 陈浩,高欣宝,李天鹏,等.国外烟幕干扰弹发展及关键技术研究[J].飞航导弹,2017,(12):71-74.
- [2] 徐路程,郝雪颖,肖凯涛,等.爆炸型烟幕弹遮蔽效能仿真研究[J].兵工学报,2020,41(07):1299-1306.
- [3] 张丽平.粒子群优化算法的理论及实践[D].浙江大学,2005.
- [4] 胡旺,李志蜀.一种更简化而高效的粒子群优化算法[J].软件学报,2007,(04):861-868.

本参赛队未使用任何 AI 工具

附录

支撑材料列表	
源代码	1_1.py 计算第一问遮掩时间 1_2.py 绘制图 11 1_3.py 绘制图 13 1_4.py 绘制图 12 1_5.py 绘制图 12 2_1.py 粒子群优化算法计算第二问遮掩时间 3_1.py 粒子群优化算法计算第三问遮掩时间 3_2.py 计算第三问 result1 结果 4_1.py 粒子群优化算法计算第四问遮掩时间 5_1.py 粒子群优化算法计算第五问遮掩时间
结果表格	表格 1: result1.xlsx 表格 2: result2.xlsx 表格 3: result4.xlsx

代码部分:

附录 1
1_1.py 计算第一问遮掩时间 import numpy as np #位置 P0_M=np.array([20000,0,2000]) P0_FY=np.array([17800,0,1800]) #速度 V_M=300 #导弹速度 V_F=120 #飞行器速度 V_g=3 #烟雾下降速度 g=9.8 #重力加速度 #尺度 R=7 #真目标半径

```

H=10    #真目标高度
r=10    #烟雾半径

T1=1.5  #飞行时间
T2=3.6  #引信时间
T3=20   #烟雾有效时间

def proj_to_new_frame(xyz_old, v, origin):
    w = v / np.linalg.norm(v) # 新坐标系 Z 轴方向矢量

    # 任意选择一个方向
    a = np.array([0, 0, 1])
    if abs(np.dot(a, w)) > 0.999: # 如果这个方向与 w 方向一致, 再换一个
        a = np.array([0, 1, 0])

    u = np.cross(a, w) # 得到新坐标系的 X 轴
    u = u / np.linalg.norm(u) # X 轴单位方向
    vnew = np.cross(w, u) # 新坐标系的 Y 轴方向

    B = np.column_stack([u, vnew, w]) # 新坐标系的三个基矢方向组合成矩阵

    # 投影的思想, 得到旧的坐标点在新的坐标系中的坐标值
    if xyz_old.ndim == 1:
        XYZ_new = B.T @ (xyz_old - origin)
    else:
        XYZ_new = B.T @ (xyz_old - origin.reshape(-1, 1))

    return XYZ_new

def xyz_M1_t(t):
    """导弹位置函数"""
    r_M1 = -P0_M / np.linalg.norm(P0_M) # 导弹飞行方向单位矢量
    return P0_M + r_M1 * V_M * t

def xyz_F1_t(t):

```

```

"""无人机位置函数"""
r_F1 = -np.array([P0_FY[0], P0_FY[1], 0]) / np.linalg.norm(P0_FY[:2]) # 水平飞行方向
return P0_FY + r_F1 * V_F * t

def xyz_Gan_t(t):
    """干扰弹位置函数"""
    base_pos = xyz_F1_t(t)
    if t >= T1:
        base_pos[2] = base_pos[2] - 0.5 * g * (t - T1)**2 # 垂直下降
    return base_pos

def xyz_Qiu_t(t):
    """云团中心位置函数"""
    if t < T1 + T2:
        return xyz_Gan_t(T1 + T2) # 还未爆炸
    else:
        base_pos = xyz_Gan_t(T1 + T2) # 爆炸位置
        base_pos[2] = base_pos[2] - V_g * (t - T1 - T2) # 云团下降
    return base_pos

theta_0 = np.linspace(0, 2*np.pi, 100, endpoint=False)
r_goal = 7
h_goal = 10
y_0_goal = 200

# 下圆周
x_goal_bottom = r_goal * np.cos(theta_0)
y_goal_bottom = y_0_goal + r_goal * np.sin(theta_0)
z_goal_bottom = np.zeros_like(x_goal_bottom)

# 上圆周
x_goal_up = r_goal * np.cos(theta_0)
y_goal_up = y_0_goal + r_goal * np.sin(theta_0)
z_goal_up = h_goal * np.ones_like(x_goal_bottom)

```

```

# 合并
x_goal_old = np.concatenate([x_goal_bottom, x_goal_up])
y_goal_old = np.concatenate([y_goal_bottom, y_goal_up])
z_goal_old = np.concatenate([z_goal_bottom, z_goal_up])
xyz_goal_old = np.array([x_goal_old, y_goal_old, z_goal_old])

def function1(t):

    xyz_M1_i = xyz_M1_t(t)
    xyz_Qiu_i = xyz_Qiu_t(t)

    r_Qiu_M1 = xyz_Qiu_i - xyz_M1_i # 导弹指向云团中心的矢量
    d_M1_Qiu = np.linalg.norm(r_Qiu_M1) # 导弹与云团中心的距离

    if d_M1_Qiu > r: # 导弹不在云团内
        r_norm = r_Qiu_M1 / d_M1_Qiu # 单位方向矢量
        theta = np.arcsin(r / d_M1_Qiu) # 圆锥半角

        # 坐标变换
        xyz_goal_new = proj_to_new_frame(xyz_goal_old, r_norm, xyz_M1_i)

        # 检查是否有点在圆锥视线之外
        # 圆锥条件:  $z < \sqrt{x^2 + y^2} / \tan(\theta)$ 
        rho = np.sqrt(xyz_goal_new[0]**2 + xyz_goal_new[1]**2)
        outside_cone = xyz_goal_new[2] < rho / np.tan(theta)

        if not np.any(outside_cone): # 如果没有点在圆锥外
            return True # 有效遮蔽
        else:
            return False
    else: # 导弹在云团内, 肯定有效遮蔽
        return True

if __name__ == "__main__":

```

```

dt = 0.0001
t_start = T1 + T2 # 5.1s
t_end = T1+T2+T3
t_array = np.arange(t_start, t_end, dt)

print(f"分析时间范围: {t_start:.1f}s - {t_end:.1f}s")
print(f"时间步长: {dt}s")
print(f"总时间点数: {len(t_array)}")

# 寻找有效遮蔽时间
t_eff = []
effective_count = 0

for i, t in enumerate(t_array):
    if function1(t):
        t_eff.append(t)
        effective_count += 1

if t_eff:
    total_effective_time = t_eff[-1] - t_eff[0]
    print(f"有效遮蔽时间: {total_effective_time:.3f}s")
    print(f"有效时间点数: {len(t_eff)}")
    print(f"遮蔽开始时间: {t_eff[0]:.3f}s")
    print(f"遮蔽结束时间: {t_eff[-1]:.3f}s")
    print(f"遮蔽效率: {len(t_eff)/len(t_array)*100:.1f}%")
else:
    print("没有有效遮蔽时间")

```

附录 2

2_1.py 粒子群优化算法计算第二问遮掩时间

```

import numpy as np
import random
import matplotlib.pyplot as plt

```



```

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

#位置
P0_M=np.array([20000,0,2000])
P0_FY=np.array([17800,0,1800])

#速度
V_M=300 #导弹速度
V_g=3 #烟雾下降速度
g=9.8 #重力加速度

#尺度
R=7 #真目标半径
H=10 #真目标高度
r=10 #烟雾半径

T3=20 #烟雾有效时间

def proj_to_new_frame(xyz_old, v, origin):
    w = v / np.linalg.norm(v) # 新坐标系 Z 轴方向矢量

    # 任意选择一个方向
    a = np.array([0, 0, 1])
    if abs(np.dot(a, w)) > 0.999: # 如果这个方向与 w 方向一致，再换一个
        a = np.array([0, 1, 0])

    u = np.cross(a, w) # 得到新坐标系的 X 轴
    u = u / np.linalg.norm(u) # X 轴单位方向
    vnew = np.cross(w, u) # 新坐标系的 Y 轴方向

    B = np.column_stack([u, vnew, w]) # 新坐标系的三个基矢方向组合成矩阵

    # 投影的思想，得到旧的坐标点在新的坐标系中的坐标值

```

```

if xyz_old.ndim == 1:
    XYZ_new = B.T @ (xyz_old - origin)
else:
    XYZ_new = B.T @ (xyz_old - origin.reshape(-1, 1))

return XYZ_new

def xyz_M1_t(t):
    """导弹位置函数"""
    r_M1 = -P0_M / np.linalg.norm(P0_M) # 导弹飞行方向单位矢量
    return P0_M + r_M1 * V_M * t

def xyz_F1_t(t, V_F, theta):
    """无人机位置函数"""
    # 使用角度 theta 确定飞行方向
    r_F1 = np.array([np.cos(theta), np.sin(theta), 0]) # 水平飞行方向
    return P0_FY + r_F1 * V_F * t

def xyz_Gan_t(t, V_F, theta, T1):
    """干扰弹位置函数"""
    base_pos = xyz_F1_t(t, V_F, theta)
    if t >= T1:
        base_pos[2] = base_pos[2] - 0.5 * g * (t - T1)**2 # 垂直下降
    return base_pos

def xyz_Qiu_t(t, V_F, theta, T1, T2):
    """云团中心位置函数"""
    if t < T1 + T2:
        return xyz_Gan_t(T1 + T2, V_F, theta, T1) # 还未爆炸
    else:
        base_pos = xyz_Gan_t(T1 + T2, V_F, theta, T1) # 爆炸位置
        base_pos[2] = base_pos[2] - V_g * (t - T1 - T2) # 云团下降
    return base_pos

# 目标离散化

```

```

theta_0 = np.linspace(0, 2*np.pi, 100, endpoint=False)
r_goal = 7
h_goal = 10
y_0_goal = 200

# 下圆周
x_goal_bottom = r_goal * np.cos(theta_0)
y_goal_bottom = y_0_goal + r_goal * np.sin(theta_0)
z_goal_bottom = np.zeros_like(x_goal_bottom)

# 上圆周
x_goal_up = r_goal * np.cos(theta_0)
y_goal_up = y_0_goal + r_goal * np.sin(theta_0)
z_goal_up = h_goal * np.ones_like(x_goal_bottom)

# 合并
x_goal_old = np.concatenate([x_goal_bottom, x_goal_up])
y_goal_old = np.concatenate([y_goal_bottom, y_goal_up])
z_goal_old = np.concatenate([z_goal_bottom, z_goal_up])
xyz_goal_old = np.array([x_goal_old, y_goal_old, z_goal_old])

def function1(t, V_F, theta, T1, T2):
    """有效遮蔽判断函数"""
    try:
        xyz_M1_i = xyz_M1_t(t)
        xyz_Qiu_i = xyz_Qiu_t(t, V_F, theta, T1, T2)

        r_Qiu_M1 = xyz_Qiu_i - xyz_M1_i # 导弹指向云团中心的矢量
        d_M1_Qiu = np.linalg.norm(r_Qiu_M1) # 导弹与云团中心的距离

        if d_M1_Qiu > r: # 导弹不在云团内
            if r / d_M1_Qiu > 1: # 防止 arcsin 错误
                return True

        r_norm = r_Qiu_M1 / d_M1_Qiu # 单位方向矢量

```

```

theta_cone = np.arcsin(r / d_M1_Qiu) # 圆锥半角

# 坐标变换
xyz_goal_new = proj_to_new_frame(xyz_goal_old, r_norm, xyz_M1_i)

# 检查是否有点在圆锥视线之外
rho = np.sqrt(xyz_goal_new[0]**2 + xyz_goal_new[1]**2)
outside_cone = xyz_goal_new[2] < rho / np.tan(theta_cone)

if not np.any(outside_cone): # 如果没有点在圆锥外
    return True # 有效遮蔽
else:
    return False
else: # 导弹在云团内，肯定有效遮蔽
    return True

except Exception as e:
    return False

def calculate_effective_shielding_time(params):
    """计算有效遮蔽时间的目标函数"""
    V_F, theta, T1, T2 = params

    # 检查参数边界
    if not (70 <= V_F <= 140):
        return 0.0
    if not (-np.pi <= theta <= np.pi):
        return 0.0
    if not (0 <= T1 <= 27):
        return 0.0
    if not (0 <= T2 <= 20):
        return 0.0

    dt = 0.01
    t_start = T1 + T2

```

```

t_end = T1 + T2 + T3

# 检查时间范围是否合理
if t_start >= t_end or t_start < 0:
    return 0.0

t_array = np.arange(t_start, t_end, dt)

if len(t_array) == 0:
    return 0.0

try:
    # 计算连续遮蔽时间段
    segments = []
    in_segment = False
    segment_start = None

    for t in t_array:
        is_effective = function1(t, V_F, theta, T1, T2)

        if is_effective and not in_segment:
            in_segment = True
            segment_start = t
        elif not is_effective and in_segment:
            in_segment = False
            segment_end = t - dt
            segments.append((segment_start, segment_end))

    # 处理最后一个连续段
    if in_segment:
        segments.append((segment_start, t_array[-1]))

    # 计算总有效遮蔽时间
    if segments:
        total_time = sum(end - start for start, end in segments)

```

```

        return total_time
    else:
        return 0.0

except Exception as e:
    print(f'计算错误: {e}, 参数: {params}')
    return 0.0

class Particle:
    """粒子类"""
    def __init__(self, dim, bounds):
        self.dim = dim # 维度
        self.bounds = bounds # 边界 [(min1, max1), (min2, max2), ...]

        # 随机初始化位置
        self.position = np.array([
            random.uniform(bounds[i][0], bounds[i][1]) for i in range(dim)
        ])

        # 随机初始化速度
        self.velocity = np.array([
            random.uniform(-1, 1) for _ in range(dim)
        ])

        # 个体最优
        self.best_position = self.position.copy()
        self.best_fitness = float('-inf') # 假设是最大化问题

        # 当前适应度
        self.fitness = float('-inf')

class PSO:
    """粒子群优化算法类"""
    def __init__(self,
                 objective_func, # 目标函数

```



```

        dim,          # 维度
        bounds,      # 边界
        pop_size=50,  # 种群大小
        max_iter=100, # 最大迭代次数
        w=0.95,       # 惯性权重
        c1=1.494,     # 认知系数
        c2=1.494):    # 社会系数

    self.objective_func = objective_func
    self.dim = dim
    self.bounds = bounds
    self.pop_size = pop_size
    self.max_iter = max_iter
    self.w = w
    self.c1 = c1
    self.c2 = c2

    # 初始化种群
    self.particles = [Particle(dim, bounds) for _ in range(pop_size)]

    # 全局最优
    self.global_best_position = None
    self.global_best_fitness = float('-inf')

    # 记录历史
    self.fitness_history = []

    def evaluate_fitness(self, particle):
        """评估粒子适应度"""
        fitness = self.objective_func(particle.position)
        return fitness

    def update_velocity(self, particle):
        """更新粒子速度"""
        r1 = np.random.random(self.dim)

```

```

r2 = np.random.random(self.dim)

# PSO 速度更新公式
cognitive_component = self.c1 * r1 * (particle.best_position - particle.position)
social_component = self.c2 * r2 * (self.global_best_position - particle.position)

particle.velocity = (self.w * particle.velocity +
                    cognitive_component +
                    social_component)

# 速度限制 (可选)
v_max = 2.0
particle.velocity = np.clip(particle.velocity, -v_max, v_max)

def update_position(self, particle):
    """更新粒子位置"""
    particle.position += particle.velocity

# 边界处理
for i in range(self.dim):
    if particle.position[i] < self.bounds[i][0]:
        particle.position[i] = self.bounds[i][0]
        particle.velocity[i] = 0 # 撞壁后速度置零
    elif particle.position[i] > self.bounds[i][1]:
        particle.position[i] = self.bounds[i][1]
        particle.velocity[i] = 0

def optimize(self):
    """主优化循环"""
    print("开始粒子群优化...")
    print(f"种群大小: {self.pop_size}")
    print(f"最大迭代次数: {self.max_iter}")
    print(f"搜索维度: {self.dim}")
    print("=" * 50)

```

```

        self.global_best_position = particle.position.copy()

        # 记录历史
        self.fitness_history.append(self.global_best_fitness)

        # 输出进度
        if (iteration + 1) % 10 == 0:
            print(f" 第 {iteration+1:3d} 代 : 最优适应度 = {self.global_best_fitness:.6f}")

        print("\n" + "=" * 50)
        print("优化完成!")
        print(f"最优适应度: {self.global_best_fitness:.6f}")
        print(f"最优位置: {self.global_best_position}")

        return self.global_best_position, self.global_best_fitness

def plot_convergence(self):
    """绘制收敛曲线"""
    plt.figure(figsize=(10, 6))
    plt.plot(self.fitness_history, 'b-', linewidth=2)
    plt.title("PSO 收敛曲线")
    plt.xlabel("迭代次数")
    plt.ylabel("最优适应度")
    plt.grid(True, alpha=0.3)
    plt.show()

class ShieldingPSO(PSO):
    """针对遮蔽时间优化的专用 PSO"""
    def __init__(self, shielding_func):
        # 定义搜索边界
        bounds = [
            (70, 140),      # V_F
            (-np.pi, np.pi), # theta
            (0, 27),        # T1

```

```

# 初始化评估
for particle in self.particles:
    particle.fitness = self.evaluate_fitness(particle)

# 更新个体最优
if particle.fitness > particle.best_fitness:
    particle.best_fitness = particle.fitness
    particle.best_position = particle.position.copy()

# 更新全局最优
if particle.fitness > self.global_best_fitness:
    self.global_best_fitness = particle.fitness
    self.global_best_position = particle.position.copy()

print(f"初始最优适应度: {self.global_best_fitness:.6f}")
self.fitness_history.append(self.global_best_fitness)

# 主循环
for iteration in range(self.max_iter):
    for particle in self.particles:
        # 更新速度和位置
        self.update_velocity(particle)
        self.update_position(particle)

        # 重新评估
        particle.fitness = self.evaluate_fitness(particle)

        # 更新个体最优
        if particle.fitness > particle.best_fitness:
            particle.best_fitness = particle.fitness
            particle.best_position = particle.position.copy()

        # 更新全局最优
        if particle.fitness > self.global_best_fitness:
            self.global_best_fitness = particle.fitness

```

```

        (0, 20)          # T2
    ]

    super().__init__(
        objective_func=shielding_func,
        dim=4,
        bounds=bounds,
        pop_size=50,
        max_iter=100,
        w=0.729,
        c1=1.494,
        c2=1.494
    )

    def optimize_shielding(self):
        """优化遮蔽参数"""
        best_pos, best_fit = self.optimize()

        print(f"\n 最优遮蔽参数:")
        print(f" 飞行器速度 V_F: {best_pos[0]:.3f} m/s")
        print(f" 飞行方向 theta: {best_pos[1]:.6f} rad
        ({np.degrees(best_pos[1]):.3f}°)")
        print(f" 飞行时间 T1: {best_pos[2]:.3f} s")
        print(f" 引信时间 T2: {best_pos[3]:.3f} s")
        print(f" 最优遮蔽时间: {best_fit:.6f} s")

        return best_pos, best_fit

if __name__ == "__main__":
    # 设置随机种子
    random.seed(42)
    np.random.seed(42)

    # 创建专用 PSO
    pso = ShieldingPSO(calculate_effective_shielding_time)

```

```

# 运行优化
best_params, best_time = pso.optimize_shielding()

# 绘制收敛曲线
pso.plot_convergence()

# 验证最优解
print("\n 验证最优解...")
verification_time = calculate_effective_shielding_time(best_params)
print(f"验证遮蔽时间: {verification_time:.6f} s")

```

附录 3

3_1.py 粒子群优化算法计算第三问遮掩时间

```

import numpy as np
import random
import matplotlib.pyplot as plt
from copy import deepcopy

# 设置中文字体
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

#位置
P0_M=np.array([20000,0,2000])
P0_FY=np.array([17800,0,1800])
#速度
V_M=300
V_g=3
g=9.8
#尺度
R=7
H=10
r=10

```


T3=20

```
def proj_to_new_frame(xyz_old, v, origin):
    w = v / np.linalg.norm(v)
    a = np.array([0, 0, 1])
    if abs(np.dot(a, w)) > 0.999:
        a = np.array([0, 1, 0])
    u = np.cross(a, w)
    u = u / np.linalg.norm(u)
    vnew = np.cross(w, u)
    B = np.column_stack([u, vnew, w])
    if xyz_old.ndim == 1:
        XYZ_new = B.T @ (xyz_old - origin)
    else:
        XYZ_new = B.T @ (xyz_old - origin.reshape(-1, 1))
    return XYZ_new

def xyz_M1_t(t):
    r_M1 = -P0_M / np.linalg.norm(P0_M)
    return P0_M + r_M1 * V_M * t

def xyz_F1_t(t, V_F, theta):
    r_F1 = np.array([np.cos(theta), np.sin(theta), 0])
    return P0_FY + r_F1 * V_F * t

def xyz_Gan_t(t, V_F, theta, T1):
    base_pos = xyz_F1_t(t, V_F, theta)
    if t >= T1:
        base_pos[2] = base_pos[2] - 0.5 * g * (t - T1)**2
    return base_pos

def xyz_Qiu_t(t, V_F, theta, T1, T2):
    if t < T1 + T2:
        return xyz_Gan_t(T1 + T2, V_F, theta, T1)
    else:
```

```

        base_pos = xyz_Gan_t(T1 + T2, V_F, theta, T1)
        base_pos[2] = base_pos[2] - V_g * (t - T1 - T2)
        return base_pos

# 目标离散化
theta_0 = np.linspace(0, 2*np.pi, 100, endpoint=False)
r_goal = 7
h_goal = 10
y_0_goal = 200
x_goal_bottom = r_goal * np.cos(theta_0)
y_goal_bottom = y_0_goal + r_goal * np.sin(theta_0)
z_goal_bottom = np.zeros_like(x_goal_bottom)
x_goal_up = r_goal * np.cos(theta_0)
y_goal_up = y_0_goal + r_goal * np.sin(theta_0)
z_goal_up = h_goal * np.ones_like(x_goal_bottom)
x_goal_old = np.concatenate([x_goal_bottom, x_goal_up])
y_goal_old = np.concatenate([y_goal_bottom, y_goal_up])
z_goal_old = np.concatenate([z_goal_bottom, z_goal_up])
xyz_goal_old = np.array([x_goal_old, y_goal_old, z_goal_old])

def calculate_effective_shielding_time(params):
    if len(params) == 8:
        # 如果是 PSO 传入的顺序 (V_F, theta, dt1, dt2, dt3, dt4, dt5, dt6)
        V_F, theta, dt1, dt2, dt3, dt4, dt5, dt6 = params
    else:
        return 0.0
    if not (70 <= V_F <= 140 and 0 <= theta <= 0.2 and
            0 <= dt1 <= 10 and 0 <= dt2 <= 10 and
            0 <= dt3 <= 10 and 0 <= dt4 <= 10 and
            0 <= dt5 <= 10 and 0 <= dt6 <= 10 and
            dt3 - dt1 >= 1 and dt5 - dt3 >= 1):
        return 0.0

    # 计算三个云团的出现时间并排序
    t_qiu = np.array([dt1 + dt2, dt3 + dt4, dt5 + dt6])

```

```

order = np.argsort(t_qiu) # 获取排序索引
t_qiu_sorted = t_qiu[order]

t_gan = np.array([dt1, dt3, dt5])

# 时间范围设置 - 参考 MATLAB 实现
dt = 0.01 # 精度与 MATLAB 一致
t_start = t_qiu_sorted[0] # 第一个云团出现时开始计时
t_end = min(t_qiu_sorted[2] + 10, t_qiu_sorted[0] + 19.99) # 最后一个云团起爆
20s 内有效

if t_end <= t_start:
    return 0.0

# 时间数组
t_array = np.arange(t_start, t_end, dt)
t_eff = np.zeros(len(t_array)) # 有效遮蔽标记数组

total_effective_time = 0.0

try:
    for i, t in enumerate(t_array):
        # 计算当前时刻各个云团的状态
        is_effective = False

        # 根据时间段判断有几个云团存在
        active_clouds = np.sum(t >= t_qiu_sorted)

        if active_clouds == 1:
            # 只有第一个云团
            cloud_idx = order[0]
            if _check_single_cloud_coverage(t, V_F, theta, t_gan[cloud_idx],
t_qiu[cloud_idx]):
                is_effective = True

```

```

elif active_clouds == 2:
    # 有两个云团
    cloud1_idx, cloud2_idx = order[0], order[1]
    if _check_two_clouds_coverage(t, V_F, theta,
                                  t_gan[cloud1_idx],
t_qiu[cloud1_idx],
                                  t_gan[cloud2_idx],
t_qiu[cloud2_idx]):
        is_effective = True

elif active_clouds == 3:
    # 三个云团都存在
    cloud1_idx, cloud2_idx, cloud3_idx = order[0], order[1], order[2]
    if _check_three_clouds_coverage(t, V_F, theta,
                                    t_gan[cloud1_idx],
t_qiu[cloud1_idx],
                                    t_gan[cloud2_idx],
t_qiu[cloud2_idx],
                                    t_gan[cloud3_idx],
t_qiu[cloud3_idx]):
        is_effective = True

    if t - t_qiu_sorted[0] >= 20:
        break # 云团 1 失效，停止计算

    if is_effective:
        t_eff[i] = 1
        total_effective_time += dt

except Exception as e:
    return 0.0

return total_effective_time

```

```

def _check_single_cloud_coverage(t, V_F, theta, t_gan, t_qiu):
    """检查单个云团的遮蔽效果"""
    try:
        xyz_M1_i = xyz_M1_t(t)
        xyz_Qiu_i = xyz_Qiu_t(t, V_F, theta, t_gan, t_qiu - t_gan)

        r_Qiu_M1 = xyz_Qiu_i - xyz_M1_i
        d_M1_Qiu = np.linalg.norm(r_Qiu_M1)

        if d_M1_Qiu <= r: # 导弹在云团内
            return True

        # 圆锥遮蔽检查
        if r / d_M1_Qiu >= 1:
            return True

        theta_cone = np.arcsin(r / d_M1_Qiu)
        r_norm = r_Qiu_M1 / d_M1_Qiu
        xyz_goal_new = proj_to_new_frame(xyz_goal_old, r_norm, xyz_M1_i)

        rho = np.sqrt(xyz_goal_new[0]**2 + xyz_goal_new[1]**2)
        outside_cone = xyz_goal_new[2] < rho / np.tan(theta_cone)

        return not np.any(outside_cone)

    except Exception:
        return False

def _check_two_clouds_coverage(t, V_F, theta, t_gan1, t_qiu1, t_gan2, t_qiu2):
    """检查两个云团的联合遮蔽效果"""
    try:
        xyz_M1_i = xyz_M1_t(t)
        xyz_Qiu_i_1 = xyz_Qiu_t(t, V_F, theta, t_gan1, t_qiu1 - t_gan1)
        xyz_Qiu_i_2 = xyz_Qiu_t(t, V_F, theta, t_gan2, t_qiu2 - t_gan2)

```

```

r_Qiu_M1_1 = xyz_Qiu_i_1 - xyz_M1_i
r_Qiu_M1_2 = xyz_Qiu_i_2 - xyz_M1_i
d_M1_Qiu_1 = np.linalg.norm(r_Qiu_M1_1)
d_M1_Qiu_2 = np.linalg.norm(r_Qiu_M1_2)

# 如果导弹在任何一个云团内
if d_M1_Qiu_1 <= r or d_M1_Qiu_2 <= r:
    return True

# 两个圆锥的联合遮蔽检查
if r / d_M1_Qiu_1 >= 1 or r / d_M1_Qiu_2 >= 1:
    return True

theta_cone_1 = np.arcsin(r / d_M1_Qiu_1)
theta_cone_2 = np.arcsin(r / d_M1_Qiu_2)

r_norm_1 = r_Qiu_M1_1 / d_M1_Qiu_1
r_norm_2 = r_Qiu_M1_2 / d_M1_Qiu_2

xyz_goal_new_1 = proj_to_new_frame(xyz_goal_old, r_norm_1, xyz_M1_i)
xyz_goal_new_2 = proj_to_new_frame(xyz_goal_old, r_norm_2, xyz_M1_i)

rho_1 = np.sqrt(xyz_goal_new_1[0]**2 + xyz_goal_new_1[1]**2)
rho_2 = np.sqrt(xyz_goal_new_2[0]**2 + xyz_goal_new_2[1]**2)

inside_cone_1 = xyz_goal_new_1[2] >= rho_1 / np.tan(theta_cone_1)
inside_cone_2 = xyz_goal_new_2[2] >= rho_2 / np.tan(theta_cone_2)

# 检查两个圆锥的并集是否完全覆盖目标
covered_points = np.logical_or(inside_cone_1, inside_cone_2)
return np.all(covered_points)

except Exception:
    return False

```



```

def _check_three_clouds_coverage(t, V_F, theta, t_gan1, t_qiu1, t_gan2, t_qiu2, t_gan3,
t_qiu3):
    """检查三个云团的联合遮蔽效果"""
    try:
        xyz_M1_i = xyz_M1_t(t)
        xyz_Qiu_i_1 = xyz_Qiu_t(t, V_F, theta, t_gan1, t_qiu1 - t_gan1)
        xyz_Qiu_i_2 = xyz_Qiu_t(t, V_F, theta, t_gan2, t_qiu2 - t_gan2)
        xyz_Qiu_i_3 = xyz_Qiu_t(t, V_F, theta, t_gan3, t_qiu3 - t_gan3)

        r_Qiu_M1_1 = xyz_Qiu_i_1 - xyz_M1_i
        r_Qiu_M1_2 = xyz_Qiu_i_2 - xyz_M1_i
        r_Qiu_M1_3 = xyz_Qiu_i_3 - xyz_M1_i
        d_M1_Qiu_1 = np.linalg.norm(r_Qiu_M1_1)
        d_M1_Qiu_2 = np.linalg.norm(r_Qiu_M1_2)
        d_M1_Qiu_3 = np.linalg.norm(r_Qiu_M1_3)

        # 如果导弹在任何一个云团内
        if d_M1_Qiu_1 <= r or d_M1_Qiu_2 <= r or d_M1_Qiu_3 <= r:
            return True

        # 三个圆锥的联合遮蔽检查
        if (r / d_M1_Qiu_1 >= 1 or r / d_M1_Qiu_2 >= 1 or r / d_M1_Qiu_3 >= 1):
            return True

        theta_cone_1 = np.arcsin(r / d_M1_Qiu_1)
        theta_cone_2 = np.arcsin(r / d_M1_Qiu_2)
        theta_cone_3 = np.arcsin(r / d_M1_Qiu_3)

        r_norm_1 = r_Qiu_M1_1 / d_M1_Qiu_1
        r_norm_2 = r_Qiu_M1_2 / d_M1_Qiu_2
        r_norm_3 = r_Qiu_M1_3 / d_M1_Qiu_3

        xyz_goal_new_1 = proj_to_new_frame(xyz_goal_old, r_norm_1, xyz_M1_i)

```

```

xyz_goal_new_2 = proj_to_new_frame(xyz_goal_old, r_norm_2, xyz_M1_i)
xyz_goal_new_3 = proj_to_new_frame(xyz_goal_old, r_norm_3, xyz_M1_i)

rho_1 = np.sqrt(xyz_goal_new_1[0]**2 + xyz_goal_new_1[1]**2)
rho_2 = np.sqrt(xyz_goal_new_2[0]**2 + xyz_goal_new_2[1]**2)
rho_3 = np.sqrt(xyz_goal_new_3[0]**2 + xyz_goal_new_3[1]**2)

inside_cone_1 = xyz_goal_new_1[2] >= rho_1 / np.tan(theta_cone_1)
inside_cone_2 = xyz_goal_new_2[2] >= rho_2 / np.tan(theta_cone_2)
inside_cone_3 = xyz_goal_new_3[2] >= rho_3 / np.tan(theta_cone_3)

# 检查三个圆锥的并集是否完全覆盖目标
covered_points = np.logical_or(np.logical_or(inside_cone_1, inside_cone_2),
inside_cone_3)
return np.all(covered_points)

except Exception:
    return False

class TS_Particle:
    """时序粒子类 - 支持 pbest 和 gbest 的时间扰动机制"""
    def __init__(self, dim, bounds):
        self.dim = dim
        self.bounds = bounds

        # 生成满足约束的初始位置
        self.position = self._generate_valid_position()
        self.velocity = np.random.uniform(-0.1, 0.1, dim)

        # 个体最优
        self.pbest_position = self.position.copy()
        self.pbest_fitness = float('-inf')
        self.pbest_age = 0 # pbest 停滞时间

        # 当前适应度

```

```

self.current_fitness = float('-inf')

def _generate_valid_position(self):
    """生成满足约束条件的有效位置"""
    max_attempts = 100
    for _ in range(max_attempts):
        position = np.array([random.uniform(self.bounds[i][0], self.bounds[i][1])
for i in range(self.dim)])
        if self.dim == 8:
            V_F, theta, dt1, dt2, dt3, dt4, dt5, dt6 = position
            if dt3 - dt1 >= 1 and dt5 - dt3 >= 1:
                return position
        else:
            return position

# 如果随机生成失败，手动构造满足约束的解
if self.dim == 8:
    position = np.array([random.uniform(b[0], b[1]) for b in self.bounds])
    dt1 = position[2]
    dt3 = dt1 + random.uniform(1, 5)
    dt5 = dt3 + random.uniform(1, 5)
    position[4] = np.clip(dt3, self.bounds[4][0], self.bounds[4][1])
    position[6] = np.clip(dt5, self.bounds[6][0], self.bounds[6][1])
    return position
return np.array([random.uniform(b[0], b[1]) for b in self.bounds])

class TSPSO_Optimizer:
    """单种群时序粒子群优化算法 (tsPSO)"""

    def __init__(self, objective_func, dim, bounds, pop_size=50, max_iter=100,
                 w=0.8, c1=2.0, c2=2.0, T0=3, Tg=5, use_dynamic=False):
        """
        初始化 tsPSO 优化器

```

参数:

- objective_func: 目标函数
- dim: 问题维度
- bounds: 变量边界 [(min1, max1), (min2, max2), ...]
- pop_size: 种群大小
- max_iter: 最大迭代次数
- w: 惯性权重
- c1, c2: 学习因子
- T0: pbest 扰动阈值
- Tg: gbest 扰动阈值
- use_dynamic: 是否使用动态参数

"""

```
self.objective_func = objective_func
self.dim = dim
self.bounds = np.array(bounds)
self.pop_size = pop_size
self.max_iter = max_iter
```

tsPSO 核心参数

```
self.w = w
self.c1 = c1
self.c2 = c2
self.T0 = T0
self.Tg = Tg
self.use_dynamic = use_dynamic
```

初始化种群

```
self.particles = [TS_Particle(dim, bounds) for _ in range(pop_size)]
```

全局最优

```
self.gbest_position = None
self.gbest_fitness = float('-inf')
self.gbest_age = 0 # gbest 停滞时间
```

记录

```

self.fitness_history = []
self.diversity_history = []
self.w_history = []
self.c1_history = []
self.c2_history = []
self.T0_history = []
self.Tg_history = []

# 动态参数的初始值
self.w_initial = w
self.c1_initial = c1
self.c2_initial = c2
self.T0_initial = T0
self.Tg_initial = Tg

def _enforce_time_constraints(self, particle):
    """强制粒子位置满足问题的时间约束"""
    if self.dim != 8:
        return

    V_F, theta, dt1, dt2, dt3, dt4, dt5, dt6 = particle.position

    # 约束 1: dt3 - dt1 >= 1
    if dt3 < dt1 + 1:
        particle.position[4] = min(dt1 + 1, self.bounds[4, 1])

    # 更新 dt3 为修正后的值
    dt3 = particle.position[4]

    # 约束 2: dt5 - dt3 >= 1
    if dt5 < dt3 + 1:
        particle.position[6] = min(dt3 + 1, self.bounds[6, 1])

def _calculate_diversity(self):
    """计算种群多样性"""

```

```

if len(self.particles) < 2:
    return 0.0

positions = np.array([p.position for p in self.particles])
mean_pos = np.mean(positions, axis=0)
diversity = np.mean([np.linalg.norm(pos - mean_pos) for pos in positions])
return diversity

def _update_dynamic_parameters(self, iteration):
    """动态更新算法参数"""
    if not self.use_dynamic:
        return

    # 计算进度百分比
    progress = iteration / self.max_iter
    diversity = self._calculate_diversity()

    # 1. 线性递减惯性权重
    self.w = self.w_initial - (self.w_initial - 0.4) * progress

    # 2. 动态学习因子
    self.c1 = self.c1_initial - 0.5 * progress # 个体学习递减
    self.c2 = self.c2_initial + 0.5 * progress # 群体学习递增

    # 3. 自适应扰动阈值
    base_T0 = max(1, self.T0_initial - int(3 * progress))
    base_Tg = max(1, self.Tg_initial - int(3 * progress))

    # 基于多样性调整
    if diversity < 0.1: # 多样性过低, 增强扰动
        self.T0 = max(1, base_T0 - 1)
        self.Tg = max(1, base_Tg - 1)
    else:
        self.T0 = base_T0
        self.Tg = base_Tg

```



```

# 基于停滞情况调整
if self.gbest_age > 20: # 全局最优停滞过久
    self.T0 = max(1, self.T0 - 1)
    self.Tg = max(1, self.Tg - 1)

# 记录参数变化
self.w_history.append(self.w)
self.c1_history.append(self.c1)
self.c2_history.append(self.c2)
self.T0_history.append(self.T0)
self.Tg_history.append(self.Tg)

def optimize(self):
    """执行 tsPSO 优化"""
    print("开始执行单种群 tsPSO 优化...")
    print(f"种群大小: {self.pop_size}, 最大迭代次数: {self.max_iter}")
    print(f"动态参数: {'启用' if self.use_dynamic else '禁用'}")
    print("=" * 70)

    # 初始化评估
    for particle in self.particles:
        particle.current_fitness = self.objective_func(particle.position)
        if particle.current_fitness > particle.pbest_fitness:
            particle.pbest_fitness = particle.current_fitness
            particle.pbest_position = particle.position.copy()

        if particle.current_fitness > self.gbest_fitness:
            self.gbest_fitness = particle.current_fitness
            self.gbest_position = particle.position.copy()
            self.gbest_age = 0

    # 主循环
    for iteration in range(self.max_iter):
        # 动态参数更新

```

```

self._update_dynamic_parameters(iteration)

gbest_updated_in_iter = False

# 更新每个粒子
for particle in self.particles:
    # 评估当前位置
    particle.current_fitness = self.objective_func(particle.position)

    # 更新个体最优
    if particle.current_fitness > particle.pbest_fitness:
        particle.pbest_fitness = particle.current_fitness
        particle.pbest_position = particle.position.copy()
        particle.pbest_age = 0
    else:
        particle.pbest_age += 1

    # 更新全局最优
    if particle.current_fitness > self.gbest_fitness:
        self.gbest_fitness = particle.current_fitness
        self.gbest_position = particle.position.copy()
        gbest_updated_in_iter = True

# 更新 gbest 年龄
if gbest_updated_in_iter:
    self.gbest_age = 0
else:
    self.gbest_age += 1

# 更新粒子位置和速度
for particle in self.particles:
    # tsPSO 核心：基于年龄的扰动机制
    r3 = random.random() if particle.pbest_age > self.T0 else 1.0
    r4 = random.random() if self.gbest_age > self.Tg else 1.0

```

```

# 应用扰动
disturbed_pbest = r3 * particle.pbest_position
disturbed_gbest = r4 * self.gbest_position

# 标准 PSO 速度更新公式
r1 = np.random.rand(self.dim)
r2 = np.random.rand(self.dim)

particle.velocity = (self.w * particle.velocity +
                    self.c1 * r1 * (disturbed_pbest -
particle.position) +
                    self.c2 * r2 * (disturbed_gbest -
particle.position))

# 位置更新
particle.position = particle.position + particle.velocity

# 边界约束
np.clip(particle.position, self.bounds[:, 0], self.bounds[:, 1],
out=particle.position)

# 时间约束
self.enforce_time_constraints(particle)

# 记录历史
self.fitness_history.append(self.gbest_fitness)
diversity = self.calculate_diversity()
self.diversity_history.append(diversity)

# 进度输出
if (iteration + 1) % 10 == 0:
    print(f" 第 {iteration+1:3d} 代 : 最优遮蔽时间 =
{self.gbest_fitness:.6f} s, "
          f"多样性 = {diversity:.4f}")
    if self.use_dynamic:

```

```

        print(f"        参数 : w={self.w:.3f}, c1={self.c1:.3f},
c2={self.c2:.3f}, "
              f"T0={self.T0}, Tg={self.Tg}")

    print("\n" + "=" * 70)
    print("tsPSO 优化完成!")
    return self.gbest_position, self.gbest_fitness

def plot_convergence(self):

    plt.figure(figsize=(10, 6))
    plt.plot(self.fitness_history, color=(0.2, 0.6, 0.8), linewidth=2)
    plt.xlabel('迭代次数')
    plt.ylabel('最优适应度')
    plt.grid(True, alpha=0.3)
    plt.show()

if __name__ == "__main__":
    # 设置随机种子
    random.seed(666)
    np.random.seed(666)

    # 定义参数边界
    bounds = [
        (70, 140), # v: 无人机速度
        (0, 0.2), # theta_v: 飞行方向角
        (0, 10),  # deltat1: 第一颗烟雾弹飞行时间
        (0, 10),  # deltat2: 第一颗烟雾弹引信时间
        (0, 10),  # deltat3: 第二颗烟雾弹飞行时间
        (0, 10),  # deltat4: 第二颗烟雾弹引信时间
        (0, 10),  # deltat5: 第三颗烟雾弹飞行时间
        (0, 10),  # deltat6: 第三颗烟雾弹引信时间
    ]

```

```

# 创建 tsPSO 优化器
ts_pso = TSPSO_Optimizer(
    objective_func=calculate_effective_shielding_time,
    dim=8,
    bounds=bounds,
    pop_size=100,      # 单种群可以设置更大的种群规模
    max_iter=30,      # 更多的迭代次数
    w=0.9,            # 初始惯性权重
    c1=2.5,           # 个体学习因子
    c2=1.5,           # 群体学习因子
    T0=3,             # pbest 扰动阈值
    Tg=5,             # gbest 扰动阈值
    use_dynamic=True  # 启用动态参数策略
)

# 执行优化
best_params, best_time = ts_pso.optimize()

# 输出结果
print(f"\n 最优三烟雾弹遮蔽参数 (单种群 tsPSO):")
print(f" 飞行器速度 V_F: {best_params[0]:.3f} m/s")
print(f" 飞行方向 theta: {best_params[1]:.6f} rad
({np.degrees(best_params[1]):.3f}°)")
print(f" 第一颗烟雾弹 - 飞行时间 dt1: {best_params[2]:.3f} s")
print(f" 第一颗烟雾弹 - 引信时间 dt2: {best_params[3]:.3f} s")
print(f" 第二颗烟雾弹 - 飞行时间 dt3: {best_params[4]:.3f} s")
print(f" 第二颗烟雾弹 - 引信时间 dt4: {best_params[5]:.3f} s")
print(f" 第三颗烟雾弹 - 飞行时间 dt5: {best_params[6]:.3f} s")
print(f" 第三颗烟雾弹 - 引信时间 dt6: {best_params[7]:.3f} s")
print(f" 最优遮蔽时间: {best_time:.6f} s")

# 约束条件验证
dt1, dt3, dt5 = best_params[2], best_params[4], best_params[6]
print(f"\n 约束条件验证:")
print(f" dt3 - dt1 = {dt3 - dt1:.3f} s (要求 ≥ 1.0 s) - {'满足' if dt3 - dt1 >= 1.0 else

```

```

'不满足'})
    print(f" dt5 - dt3 = {dt5 - dt3:.3f} s (要求  $\geq 1.0$  s) - {'满足' if dt5 - dt3 >= 1.0 else '不满足'})

    # 计算关键位置信息
    V_F, theta, dt1, dt2, dt3, dt4, dt5, dt6 = best_params

    print(f"\n 关键时间节点:")
    print(f"  第一颗烟雾弹起爆时间: {dt1 + dt2:.3f} s")
    print(f"  第二颗烟雾弹起爆时间: {dt3 + dt4:.3f} s")
    print(f"  第三颗烟雾弹起爆时间: {dt5 + dt6:.3f} s")

    # 计算三个烟雾干扰弹的详细位置信息
    print(f"\n 烟雾干扰弹投放点坐标:")

    # 第一颗烟雾弹投放点 (t=dt1 时的飞行器位置)
    release_pos_1 = xyz_F1_t(dt1, V_F, theta)
    print(f"  第一颗烟雾弹投放点: ({release_pos_1[0]:.1f}, {release_pos_1[1]:.1f}, {release_pos_1[2]:.1f}) m")

    # 第二颗烟雾弹投放点 (t=dt3 时的飞行器位置)
    release_pos_2 = xyz_F1_t(dt3, V_F, theta)
    print(f"  第二颗烟雾弹投放点: ({release_pos_2[0]:.1f}, {release_pos_2[1]:.1f}, {release_pos_2[2]:.1f}) m")

    # 第三颗烟雾弹投放点 (t=dt5 时的飞行器位置)
    release_pos_3 = xyz_F1_t(dt5, V_F, theta)
    print(f"  第三颗烟雾弹投放点: ({release_pos_3[0]:.1f}, {release_pos_3[1]:.1f}, {release_pos_3[2]:.1f}) m")

    print(f"\n 烟雾干扰弹起爆点坐标:")

    # 第一颗烟雾弹起爆点 (t=dt1+dt2 时的干扰弹位置)
    explosion_pos_1 = xyz_Gan_t(dt1 + dt2, V_F, theta, dt1)
    print(f"  第一颗烟雾弹起爆点: ({explosion_pos_1[0]:.1f},

```

```

{explosion_pos_1[1]:.1f}, {explosion_pos_1[2]:.1f}) m")

# 第二颗烟雾弹起爆点 (t=dt3+dt4 时的干扰弹位置)
explosion_pos_2 = xyz_Gan_t(dt3 + dt4, V_F, theta, dt3)
print(f"    第二颗烟雾弹起爆点 : ({explosion_pos_2[0]:.1f},
{explosion_pos_2[1]:.1f}, {explosion_pos_2[2]:.1f}) m")

# 第三颗烟雾弹起爆点 (t=dt5+dt6 时的干扰弹位置)
explosion_pos_3 = xyz_Gan_t(dt5 + dt6, V_F, theta, dt5)
print(f"    第三颗烟雾弹起爆点 : ({explosion_pos_3[0]:.1f},
{explosion_pos_3[1]:.1f}, {explosion_pos_3[2]:.1f}) m")

print(f"\n 烟雾干扰弹有效时长:")

# 显示收敛分析图
ts_pso.plot_convergence()

```

附录 4

4_1.py 粒子群优化算法计算第四问遮掩时间

```

import numpy as np
import random
import matplotlib.pyplot as plt

# 设置中文字体, 确保图片能正确显示中文
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 位置
P0_M = np.array([20000, 0, 2000])
P0_FY_1 = np.array([17800, 0, 1800])
P0_FY_2 = np.array([12000, 1400, 1400])
P0_FY_3 = np.array([6000, -3000, 700])

# 速度

```



```

V_M = 300
V_g = 3
g = 9.8
# 尺度
R = 7
H = 10
r = 10
T3 = 20

#坐标洗变换
def proj_to_new_frame(xyz_old, v, origin):
    w = v / np.linalg.norm(v)
    a = np.array([0, 0, 1])
    if abs(np.dot(a, w)) > 0.999:
        a = np.array([0, 1, 0])
    u = np.cross(a, w)
    u = u / np.linalg.norm(u)
    vnew = np.cross(w, u)
    B = np.column_stack([u, vnew, w])
    if xyz_old.ndim == 1:
        XYZ_new = B.T @ (xyz_old - origin)
    else:
        XYZ_new = B.T @ (xyz_old - origin.reshape(-1, 1))
    return XYZ_new

def xyz_M1_t(t):
    r_M1 = -P0_M / np.linalg.norm(P0_M)
    return P0_M + r_M1 * V_M * t

def xyz_F1_t(t, V_F, theta, P0_FY):
    r_F1 = np.array([np.cos(theta), np.sin(theta), 0])
    return P0_FY + r_F1 * V_F * t

def xyz_Gan_t(t, V_F, theta, T1, P0_FY):
    base_pos = xyz_F1_t(t, V_F, theta, P0_FY)

```

```

    if t >= T1:
        base_pos[2] = base_pos[2] - 0.5 * g * (t - T1)**2
    return base_pos

def xyz_Qiu_t(t, V_F, P0_FY, theta, T1, T2):
    if t < T1 + T2:
        return xyz_Gan_t(T1 + T2, V_F, theta, T1, P0_FY)
    else:
        base_pos = xyz_Gan_t(T1 + T2, V_F, theta, T1, P0_FY)
        base_pos[2] = base_pos[2] - V_g * (t - T1 - T2)
    return base_pos

# 目标离散化
theta_0 = np.linspace(0, 2 * np.pi, 100, endpoint=False)
r_goal = 7
h_goal = 10
y_0_goal = 300
x_goal_bottom = r_goal * np.cos(theta_0)
y_goal_bottom = y_0_goal + r_goal * np.sin(theta_0)
z_goal_bottom = np.zeros_like(x_goal_bottom)
x_goal_up = r_goal * np.cos(theta_0)
y_goal_up = y_0_goal + r_goal * np.sin(theta_0)
z_goal_up = h_goal * np.ones_like(x_goal_bottom)
x_goal_old = np.concatenate([x_goal_bottom, x_goal_up])
y_goal_old = np.concatenate([y_goal_bottom, y_goal_up])
z_goal_old = np.concatenate([z_goal_bottom, z_goal_up])
xyz_goal_old = np.array([x_goal_old, y_goal_old, z_goal_old])

def calculate_effective_shielding_time(params):
    if len(params) == 12:
        V_F_1, theta_1, dt1, dt2, V_F_2, theta_2, dt3, dt4, V_F_3, theta_3, dt5, dt6 =
params
    else:
        return 0.0
    if not (80 <= V_F_1 <= 115 and 0.08 <= theta_1 <= 0.11 and 0.55 <= dt1 <= 0.85 and

```

```

0.4 <= dt2 <= 0.55 and
    70 <= V_F_2 <= 90 and 4 <= theta_2 <= 5 and 10 <= dt3 <= 15 and 5 <=
dt4 <= 9 and
    80 <= V_F_3 <= 120 and 1.6 <= theta_3 <= 2.2 and 20 <= dt5 <= 30 and 5
<= dt6 <= 8):
    return 0.0

dt = 0.01
t_gan = np.array([dt1, dt3, dt5])
t_P0_FY = np.array([P0_FY_1, P0_FY_2, P0_FY_3])
t_theta = np.array([theta_1, theta_2, theta_3])
t_VF = np.array([V_F_1, V_F_2, V_F_3])
t_qiu = np.array([dt1 + dt2, dt3 + dt4, dt5 + dt6])

order = np.argsort(t_qiu)
t_qiu_sorted = t_qiu[order]
t_expire = t_qiu + T3

t_start = t_qiu_sorted[0]
t_end = max(t_expire) # 使用所有烟雾中最晚消失的时间

t_array = np.arange(t_start, t_end, dt)
total_effective_time = 0.0

try:
    for i, t in enumerate(t_array):
        is_effective = False
        cloud_active = np.logical_and(t >= t_qiu, t < t_expire)
        active_clouds_indices = np.where(cloud_active)[0]
        active_clouds_count = len(active_clouds_indices)

        if active_clouds_count == 1:
            cloud_idx = active_clouds_indices[0]
            if _check_single_cloud_coverage(t, t_VF[cloud_idx],
t_P0_FY[cloud_idx], t_theta[cloud_idx], t_gan[cloud_idx], t_qiu[cloud_idx]):

```

```

        is_effective = True
    elif active_clouds_count == 2:
        cloud1_idx, cloud2_idx = active_clouds_indices[0],
        active_clouds_indices[1]
        if _check_two_clouds_coverage(t, t_VF[cloud1_idx],
        t_P0_FY[cloud1_idx], t_theta[cloud1_idx], t_gan[cloud1_idx], t_qiu[cloud1_idx],
        t_VF[cloud2_idx],
        t_P0_FY[cloud2_idx], t_theta[cloud2_idx], t_gan[cloud2_idx], t_qiu[cloud2_idx]):
            is_effective = True
    elif active_clouds_count == 3:
        if _check_three_clouds_coverage(t, t_VF[0], t_P0_FY[0], t_theta[0],
        t_gan[0], t_qiu[0],
        t_VF[1], t_P0_FY[1],
        t_theta[1], t_gan[1], t_qiu[1],
        t_VF[2], t_P0_FY[2],
        t_theta[2], t_gan[2], t_qiu[2]):
            is_effective = True

    if is_effective:
        total_effective_time += dt

except Exception as e:
    return 0.0

return total_effective_time

def _check_single_cloud_coverage(t, V_F, P0_FY, theta, t_gan, t_qiu):
    try:
        xyz_M1_i = xyz_M1_t(t)
        xyz_Qiu_i = xyz_Qiu_t(t, V_F, P0_FY, theta, t_gan, t_qiu - t_gan)
        r_Qiu_M1 = xyz_Qiu_i - xyz_M1_i
        d_M1_Qiu = np.linalg.norm(r_Qiu_M1)
        if d_M1_Qiu <= r:
            return True
    
```

```

        if r / d_M1_Qiu >= 1:
            return True
        theta_cone = np.arcsin(r / d_M1_Qiu)
        r_norm = r_Qiu_M1 / d_M1_Qiu
        xyz_goal_new = proj_to_new_frame(xyz_goal_old, r_norm, xyz_M1_i)
        rho = np.sqrt(xyz_goal_new[0]**2 + xyz_goal_new[1]**2)
        outside_cone = xyz_goal_new[2] < rho / np.tan(theta_cone)
        return not np.any(outside_cone)
    except Exception:
        return False

def _check_two_clouds_coverage(t, t_VF_1, t_P0_FY_1, t_theta_1, t_gan1, t_qiu1,
                                t_VF_2, t_P0_FY_2, t_theta_2, t_gan2, t_qiu2):
    try:
        xyz_M1_i = xyz_M1_t(t)
        xyz_Qiu_i_1 = xyz_Qiu_t(t, t_VF_1, t_P0_FY_1, t_theta_1, t_gan1, t_qiu1 -
t_gan1)
        xyz_Qiu_i_2 = xyz_Qiu_t(t, t_VF_2, t_P0_FY_2, t_theta_2, t_gan2, t_qiu2 -
t_gan2)
        r_Qiu_M1_1 = xyz_Qiu_i_1 - xyz_M1_i
        r_Qiu_M1_2 = xyz_Qiu_i_2 - xyz_M1_i
        d_M1_Qiu_1 = np.linalg.norm(r_Qiu_M1_1)
        d_M1_Qiu_2 = np.linalg.norm(r_Qiu_M1_2)
        if d_M1_Qiu_1 <= r or d_M1_Qiu_2 <= r:
            return True
        if r / d_M1_Qiu_1 >= 1 or r / d_M1_Qiu_2 >= 1:
            return True
        theta_cone_1 = np.arcsin(r / d_M1_Qiu_1)
        theta_cone_2 = np.arcsin(r / d_M1_Qiu_2)
        r_norm_1 = r_Qiu_M1_1 / d_M1_Qiu_1
        r_norm_2 = r_Qiu_M1_2 / d_M1_Qiu_2
        xyz_goal_new_1 = proj_to_new_frame(xyz_goal_old, r_norm_1, xyz_M1_i)
        xyz_goal_new_2 = proj_to_new_frame(xyz_goal_old, r_norm_2, xyz_M1_i)
        rho_1 = np.sqrt(xyz_goal_new_1[0]**2 + xyz_goal_new_1[1]**2)

```

```

rho_2 = np.sqrt(xyz_goal_new_2[0]**2 + xyz_goal_new_2[1]**2)
inside_cone_1 = xyz_goal_new_1[2] >= rho_1 / np.tan(theta_cone_1)
inside_cone_2 = xyz_goal_new_2[2] >= rho_2 / np.tan(theta_cone_2)
covered_points = np.logical_or(inside_cone_1, inside_cone_2)
return np.all(covered_points)
except Exception:
    return False

def _check_three_clouds_coverage(t, t_VF_1, t_P0_FY_1, t_theta_1, t_gan1, t_qiu1,
                                t_VF_2, t_P0_FY_2, t_theta_2, t_gan2,
                                t_qiu2,
                                t_VF_3, t_P0_FY_3, t_theta_3, t_gan3,
                                t_qiu3):
    try:
        xyz_M1_i = xyz_M1_t(t)
        xyz_Qiu_i_1 = xyz_Qiu_t(t, t_VF_1, t_P0_FY_1, t_theta_1, t_gan1, t_qiu1 -
                                t_gan1)
        xyz_Qiu_i_2 = xyz_Qiu_t(t, t_VF_2, t_P0_FY_2, t_theta_2, t_gan2, t_qiu2 -
                                t_gan2)
        xyz_Qiu_i_3 = xyz_Qiu_t(t, t_VF_3, t_P0_FY_3, t_theta_3, t_gan3, t_qiu3 -
                                t_gan3)
        r_Qiu_M1_1 = xyz_Qiu_i_1 - xyz_M1_i
        r_Qiu_M1_2 = xyz_Qiu_i_2 - xyz_M1_i
        r_Qiu_M1_3 = xyz_Qiu_i_3 - xyz_M1_i
        d_M1_Qiu_1 = np.linalg.norm(r_Qiu_M1_1)
        d_M1_Qiu_2 = np.linalg.norm(r_Qiu_M1_2)
        d_M1_Qiu_3 = np.linalg.norm(r_Qiu_M1_3)
        if d_M1_Qiu_1 <= r or d_M1_Qiu_2 <= r or d_M1_Qiu_3 <= r:
            return True
        if (r / d_M1_Qiu_1 >= 1 or r / d_M1_Qiu_2 >= 1 or r / d_M1_Qiu_3 >= 1):
            return True
        theta_cone_1 = np.arcsin(r / d_M1_Qiu_1)
        theta_cone_2 = np.arcsin(r / d_M1_Qiu_2)
        theta_cone_3 = np.arcsin(r / d_M1_Qiu_3)

```

```

r_norm_1 = r_Qiu_M1_1 / d_M1_Qiu_1
r_norm_2 = r_Qiu_M1_2 / d_M1_Qiu_2
r_norm_3 = r_Qiu_M1_3 / d_M1_Qiu_3
xyz_goal_new_1 = proj_to_new_frame(xyz_goal_old, r_norm_1, xyz_M1_i)
xyz_goal_new_2 = proj_to_new_frame(xyz_goal_old, r_norm_2, xyz_M1_i)
xyz_goal_new_3 = proj_to_new_frame(xyz_goal_old, r_norm_3, xyz_M1_i)
rho_1 = np.sqrt(xyz_goal_new_1[0]**2 + xyz_goal_new_1[1]**2)
rho_2 = np.sqrt(xyz_goal_new_2[0]**2 + xyz_goal_new_2[1]**2)
rho_3 = np.sqrt(xyz_goal_new_3[0]**2 + xyz_goal_new_3[1]**2)
inside_cone_1 = xyz_goal_new_1[2] >= rho_1 / np.tan(theta_cone_1)
inside_cone_2 = xyz_goal_new_2[2] >= rho_2 / np.tan(theta_cone_2)
inside_cone_3 = xyz_goal_new_3[2] >= rho_3 / np.tan(theta_cone_3)
covered_points = np.logical_or(np.logical_or(inside_cone_1, inside_cone_2),
inside_cone_3)
    return np.all(covered_points)
except Exception:
    return False

class GeneticAlgorithm_Optimizer:
    """遗传算法优化器"""
    def __init__(self, pop_size=50, max_iter=200, crossover_rate=0.8,
mutation_rate=0.1, elite_size=2):
        self.pop_size = pop_size
        self.max_iter = max_iter
        self.crossover_rate = crossover_rate
        self.mutation_rate = mutation_rate
        self.elite_size = elite_size # 精英个体数量

        # 12 维参数边界
        self.bounds = [
            (80, 115), # V_F_1
            (0.08, 0.11), # theta_1
            (0.55, 0.85), # dt1
            (0.4, 0.55), # dt2
            (70, 90), # V_F_2

```



```

        (4, 5),      # theta_2
        (10, 15),   # dt3
        (5, 9),     # dt4
        (80, 120),  # V_F_3
        (1.6, 2.2), # theta_3
        (20, 30),   # dt5
        (5, 8)      # dt6
    ]
    self.dim = len(self.bounds)

    # 初始化种群
    self.population = self._initialize_population()

    # 全局最优
    self.gbest_position = None
    self.gbest_fitness = float('-inf')

    # 记录优化过程
    self.best_fitness_history = []

    def _initialize_population(self):
        """在边界内随机生成初始种群"""
        population = []
        for _ in range(self.pop_size):
            individual = np.array([random.uniform(b[0], b[1]) for b in self.bounds])
            population.append(individual)
        return population

    def _tournament_selection(self, fitnesses, k=3):
        """锦标赛选择"""
        selection_ix = np.random.randint(len(self.population), size=k)
        best_ix = -1
        best_fitness = -1
        for ix in selection_ix:
            if fitnesses[ix] > best_fitness:

```

```

        best_fitness = fitnesses[ix]
        best_ix = ix
    return self.population[best_ix]

def _crossover(self, parent1, parent2):
    """单点交叉"""
    child1, child2 = parent1.copy(), parent2.copy()
    if random.random() < self.crossover_rate:
        # 随机选择一个交叉点
        pt = random.randint(1, self.dim - 1)
        # 进行交叉
        child1 = np.concatenate((parent1[:pt], parent2[pt:]))
        child2 = np.concatenate((parent2[:pt], parent1[pt:]))
    return child1, child2

def _mutate(self, individual):
    """变异操作 (随机重置)"""
    for i in range(self.dim):
        if random.random() < self.mutation_rate:
            individual[i] = random.uniform(self.bounds[i][0], self.bounds[i][1])

def optimize(self):
    """执行优化过程"""
    print("开始 GA 优化...")
    print(f"参数设置: 种群大小={self.pop_size}, 最大迭代次数={self.max_iter}, "
          f"交叉率={self.crossover_rate}, 变异率={self.mutation_rate}")
    print("-" * 50)

    for t in range(self.max_iter):
        # 1. 评估所有个体的适应度
        fitnesses = [calculate_effective_shielding_time(ind) for ind in self.population]

        # 2. 更新全局最优解

```

```

current_best_idx = np.argmax(fitnesses)
if fitnesses[current_best_idx] > self.gbest_fitness:
    self.gbest_fitness = fitnesses[current_best_idx]
    self.gbest_position = self.population[current_best_idx].copy()

# 记录历史数据
self.best_fitness_history.append(self.gbest_fitness)
avg_fitness = np.mean(fitnesses)

# 输出进度
if t % 10 == 0 or t == self.max_iter - 1:
    print(f"迭代 {t+1:3d}: 最优适应度 = {self.gbest_fitness:.6f}, "
          f"平均适应度 = {avg_fitness:.6f}")

# 3. 生成下一代种群
# 首先对当前种群按适应度排序, 方便选出精英
sorted_indices = np.argsort(fitnesses)[::-1] # 从大到小

next_gen = []

# 3.1 精英主义: 直接保留最好的个体
for i in range(self.elite_size):
    elite_idx = sorted_indices[i]
    next_gen.append(self.population[elite_idx])

# 3.2 交叉和变异
while len(next_gen) < self.pop_size:
    # 选择父代
    parent1 = self._tournament_selection(fitnesses)
    parent2 = self._tournament_selection(fitnesses)

    # 交叉
    child1, child2 = self._crossover(parent1, parent2)

    # 变异

```

```

        self._mutate(child1)
        self._mutate(child2)

        # 将子代添加到新种群中
        next_gen.append(child1)
        if len(next_gen) < self.pop_size:
            next_gen.append(child2)

        self.population = next_gen

    print("-" * 50)
    print("优化完成！")
    return self.gbest_position, self.gbest_fitness

def get_results(self):
    """获取优化结果 (与 APSO 版本相同)"""
    if self.gbest_position is None:
        return None

    V_F_1, theta_1, dt1, dt2, V_F_2, theta_2, dt3, dt4, V_F_3, theta_3, dt5, dt6 =
self.gbest_position

    results = {
        'best_fitness': self.gbest_fitness,
        'best_position': self.gbest_position,
        'parameters': {
            'V_F_1': V_F_1, 'theta_1': theta_1, 'dt1': dt1, 'dt2': dt2,
            'V_F_2': V_F_2, 'theta_2': theta_2, 'dt3': dt3, 'dt4': dt4,
            'V_F_3': V_F_3, 'theta_3': theta_3, 'dt5': dt5, 'dt6': dt6
        },
        'launch_times': [dt1, dt3, dt5],
        'explosion_times': [dt1 + dt2, dt3 + dt4, dt5 + dt6],
        'launch_coordinates': [
            xyz_F1_t(dt1, V_F_1, theta_1, dt1, P0_FY_1),
            xyz_F1_t(dt3, V_F_2, theta_2, dt3, P0_FY_2),

```

```

        xyz_F1_t(dt5, V_F_3, theta_3, dt5, P0_FY_3)
    ],
    'explosion_coordinates': [
        xyz_Gan_t(dt1 + dt2, V_F_1, theta_1, dt1, P0_FY_1),
        xyz_Gan_t(dt3 + dt4, V_F_2, theta_2, dt3, P0_FY_2),
        xyz_Gan_t(dt5 + dt6, V_F_3, theta_3, dt5, P0_FY_3)
    ]
}
return results

def plot_optimization_history(self):
    """绘制优化历史曲线 """
    plt.figure(figsize=(12, 6))

    plt.plot(range(1, len(self.best_fitness_history) + 1), self.best_fitness_history,
             color=(0.2, 0.6, 0.8), label='最优适应度 (GA)')
    plt.xlabel('迭代次数 (代)')
    plt.ylabel('适应度 (有效遮蔽时间)')
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig('GA_optimization_history.png', dpi=300, bbox_inches='tight')
    plt.show()

def analyze_cloud_coverage_details(params):
    """分析不同烟雾组合的遮掩时长"""
    if len(params) != 12:
        return {}

    V_F_1, theta_1, dt1, dt2, V_F_2, theta_2, dt3, dt4, V_F_3, theta_3, dt5, dt6 = params

    dt = 0.1
    t_gan = np.array([dt1, dt3, dt5])
    t_P0_FY = np.array([P0_FY_1, P0_FY_2, P0_FY_3])

```

```

t_theta = np.array([theta_1, theta_2, theta_3])
t_VF = np.array([V_F_1, V_F_2, V_F_3])
t_qiu = np.array([dt1 + dt2, dt3 + dt4, dt5 + dt6])
t_expire = t_qiu + T3

t_start = min(t_qiu)
t_end = max(t_expire)
t_array = np.arange(t_start, t_end, dt)

# 记录不同情况的时长
single_cloud_time = [0.0, 0.0, 0.0]
two_clouds_time = 0.0
three_clouds_time = 0.0

try:
    for t in t_array:
        cloud_active = np.logical_and(t >= t_qiu, t < t_expire)
        active_clouds_indices = np.where(cloud_active)[0]
        active_clouds_count = len(active_clouds_indices)

        is_effective = False
        if active_clouds_count == 1:
            cloud_idx = active_clouds_indices[0]
            if _check_single_cloud_coverage(t, t_VF[cloud_idx],
t_P0_FY[cloud_idx], t_theta[cloud_idx],
t_gan[cloud_idx], t_qiu[cloud_idx]):
                single_cloud_time[cloud_idx] += dt
                is_effective = True
            elif active_clouds_count == 2:
                cloud1_idx, cloud2_idx = active_clouds_indices[0],
active_clouds_indices[1]
                if _check_two_clouds_coverage(t, t_VF[cloud1_idx],
t_P0_FY[cloud1_idx], t_theta[cloud1_idx],

```

```

t_gan[cloud1_idx], t_qiu[cloud1_idx],
t_VF[cloud2_idx],
t_P0_FY[cloud2_idx],
t_theta[cloud2_idx],
t_gan[cloud2_idx], t_qiu[cloud2_idx]):
    two_clouds_time += dt
    is_effective = True
elif active_clouds_count == 3:
    if _check_three_clouds_coverage(t, t_VF[0], t_P0_FY[0], t_theta[0],
t_gan[0], t_qiu[0],
t_VF[1], t_P0_FY[1],
t_theta[1], t_gan[1], t_qiu[1],
t_VF[2], t_P0_FY[2],
t_theta[2], t_gan[2], t_qiu[2]):
        three_clouds_time += dt
        is_effective = True

except Exception as e:
    print(f"分析烟雾覆盖详情时出错: {e}")

return {
    'single_cloud_times': single_cloud_time,
    'two_clouds_time': two_clouds_time,
    'three_clouds_time': three_clouds_time,
    'total_effective_time': sum(single_cloud_time) + two_clouds_time +
three_clouds_time
}

def run_optimization_ga():
    """运行完整的优化流程 (使用遗传算法)"""
    # 创建优化器
    optimizer = GeneticAlgorithm_Optimizer(pop_size=50, max_iter=200,
crossover_rate=0.8, mutation_rate=0.1)

```



```

# 执行优化
best_position, best_fitness = optimizer.optimize()

# 获取结果
results = optimizer.get_results()

# 分析烟雾覆盖详情
# 注意：这里重新用最优参数计算一次以获得分类时长，因为 GA 在迭代中没有存储这个
coverage_details = analyze_cloud_coverage_details(best_position)

# 输出详细结果
print("\n" + "="*60)
print("优化结果详细信息 (GA)")
print("="*60)
print(f"最优有效遮蔽时间: {best_fitness:.6f} 秒")
print()

print("无人机参数:")
for i in range(3):
    V_F = results['parameters'][f'V_F_{i+1}']
    theta = results['parameters'][f'theta_{i+1}']
    dt_launch = results['parameters'][f'dt_{2*i+1}']
    dt_delay = results['parameters'][f'dt_{2*i+2}']

    print(f" 无人机 {i+1}:")
    print(f"    飞行速度 V_F_{i+1} = {V_F:.3f} m/s")
    print(f"    飞行角度 theta_{i+1} = {theta:.3f} rad")
    print(f"    ({np.degrees(theta):.1f}°)")
    print(f"    投放时间 dt_{2*i+1} = {dt_launch:.3f} s")
    print(f"    引信延时 dt_{2*i+2} = {dt_delay:.3f} s")
    print()

print("投放和起爆信息:")
for i in range(3):

```

```

        launch_coord = results['launch_coordinates'][i]
        explosion_coord = results['explosion_coordinates'][i]
        launch_time = results['launch_times'][i]
        explosion_time = results['explosion_times'][i]

        print(f" 烟雾弹 {i+1}:")
        print(f"    投放时间: {launch_time:.3f} s")
        print(f"    投 放 坐 标 : ({launch_coord[0]:.1f}, {launch_coord[1]:.1f},
{launch_coord[2]:.1f}) m")
        print(f"    起爆时间: {explosion_time:.3f} s")
        print(f"    起爆坐标: ({explosion_coord[0]:.1f}, {explosion_coord[1]:.1f},
{explosion_coord[2]:.1f}) m")
        print()

    print("烟雾遮掩时长详细分析:")
    print(f" 烟雾弹 1 单独遮掩时长: {coverage_details['single_cloud_times'][0]:.3f}
秒")
    print(f" 烟雾弹 2 单独遮掩时长: {coverage_details['single_cloud_times'][1]:.3f}
秒")
    print(f" 烟雾弹 3 单独遮掩时长: {coverage_details['single_cloud_times'][2]:.3f}
秒")
    print(f" 两个烟雾同时遮掩时长: {coverage_details['two_clouds_time']:.3f} 秒")
    print(f" 三个烟雾同时遮掩时长: {coverage_details['three_clouds_time']:.3f} 秒
")
    # 重新计算总和以避免浮点数精度问题
    total_time_from_details = sum(coverage_details['single_cloud_times']) +
coverage_details['two_clouds_time'] + coverage_details['three_clouds_time']
    print(f" 总有效遮掩时长 (详细加总): {total_time_from_details:.3f} 秒")
    print()

    # 绘制优化历史
    optimizer.plot_optimization_history()

    return results

```

```
if __name__ == "__main__":  
    # 运行优化  
    results = run_optimization_ga()
```